

Artha Drishti

AI-Powered Stock Market Intelligence Platform

Comprehensive Application Documentation

This document provides an in-depth explanation of every technology, library, framework, module, functionality, algorithm, design pattern, and software engineering concept used to build the Artha Drishti stock intelligence platform. Each section covers not just **what** is used but **why** it was chosen, **how** it works internally, and **what role** it plays in the overall system.

Attribute	Detail
Project Name	Artha Drishti (Sanskrit: Wealth Vision)
Type	Full-Stack AI-Powered Web Application
Frontend	React 18 + Vite Single-Page Application
Backend	Flask REST API + PyTorch Deep Learning Engine
Database	PostgreSQL 14+ with TimescaleDB Extension
ML Model	Multi-Target LSTM + Attention + TCN Neural Network
Screening	18+ Strategies with Committee Consensus
Risk Metrics	30+ Metrics including VaR, CVaR, Monte Carlo
License	MIT Open Source

Table of Contents

1. Application Overview and Purpose
2. High-Level System Architecture
3. Technology Stack - Backend (25+ Libraries)
4. Technology Stack - Frontend (14 Libraries)
5. Technology Stack - Infrastructure
6. Backend Module: application.py - Flask API Server
7. Backend Module: MLPredictor.py - Deep Learning Engine
8. Backend Module: StockScreener.py - Screening Engine
9. Backend Module: FeatureEngineering.py - Indicator Pipeline
10. Backend Module: IntegratedPostgreSQL.py - Data Pipeline
11. Backend Module: BacktestEngine.py - Backtesting System
12. Backend Module: RiskAnalytics.py - Risk Analysis
13. Backend Module: SentimentEngine.py - Sentiment Analysis
14. Backend Module: FundamentalAnalysis.py - Fundamental Scoring
15. Backend Module: PatternDetector.py - Chart Patterns
16. Other Backend Modules (10 Modules)
17. Frontend Components (14 Components)
18. Frontend Services, Routing, and Design System
19. Database Schema and Design
20. Authentication and Security Architecture
21. Scheduling and Automation Pipeline
22. Caching Layer and Distributed Task Queue
23. Deployment and Containerization
24. Complete API Reference (90+ Endpoints)
25. Design Patterns Applied (10 Patterns)
26. Machine Learning Concepts Explained
27. Financial Concepts and Algorithms Explained
28. Software Engineering Concepts Explained

1. Application Overview and Purpose

Artha Drishti (Sanskrit for 'Wealth Vision') is a comprehensive, full-stack stock market intelligence platform designed to bring professional-grade financial analysis tools to individual investors. The platform integrates five major pillars of stock analysis into a single unified workflow: (1) AI-driven price prediction using deep learning, (2) multi-strategy stock screening with committee consensus, (3) historical strategy backtesting with walk-forward validation, (4) quantitative risk analytics, and (5) fundamental and sentiment analysis.

The platform is built as a modern web application with a React frontend and Flask backend. It is designed to handle the Indian stock market (NSE/BSE) as its primary focus, but also supports international exchanges including NYSE, NASDAQ, and LSE. The system ingests daily market data automatically via a scheduled pipeline, computes 150+ technical indicators, trains and runs deep learning models, and presents all analytics through an interactive dark-themed dashboard.

What makes this platform unique:

- **Multi-Target AI Prediction:** Unlike traditional systems that predict a single target (e.g., price direction only), Artha Drishti's neural network simultaneously predicts five interdependent targets in a single forward pass: price direction (Bullish/Bearish/Neutral), volatility regime (Low/Medium/High), risk level (Low/Medium/High), trend strength (0.0-1.0 continuous), and optimal trading action (Strong Buy/Buy/Hold/Sell/Strong Sell). This gives investors a complete multi-dimensional view of a stock's outlook.
- **Committee-of-Experts Screening:** Instead of relying on a single screening strategy (which introduces single-strategy bias), the platform evaluates every stock against 18+ independent quantitative strategies across 7 categories simultaneously. Only stocks that achieve consensus agreement across multiple fundamentally different strategies are recommended. This is analogous to having a committee of 18 expert analysts independently evaluate each stock.
- **30+ Risk Metrics:** The risk analytics module computes an extensive suite of metrics including Value at Risk (VaR), Conditional VaR (Expected Shortfall), Sharpe/Sortino/Calmar ratios, Hurst exponent for trend persistence, Monte Carlo efficient frontier with Dirichlet sampling, and Component VaR decomposition for portfolio-level risk attribution.
- **Walk-Forward Backtesting:** The backtesting engine uses walk-forward cross-validation (expanding training window with fixed test window) to prevent look-ahead bias. It also includes automatic market regime detection (bull/bear/sideways) and shows how strategies perform differently across regimes.
- **Dual Fundamental Scoring:** Combines the academically rigorous Piotroski F-Score (9-point) with a proprietary 5-category weighted composite score (0-100) covering valuation, profitability, growth, financial health, and dividends.
- **Autonomous Report Generation:** The system can automatically generate self-contained HTML intelligence dossiers that combine AI predictions, fundamental assessments, risk metrics, and technical analysis into a single professional investor-ready document.
- **Automated Data Pipeline:** A scheduler-driven pipeline automatically fetches market data at 4:00 PM IST (Indian market close), computes all technical features, and optionally retrains ML models - all without human intervention.

2. High-Level System Architecture

The application follows a **layered client-server architecture** with clear separation of concerns. Each layer has a specific responsibility, communicates through well-defined interfaces, and can be scaled or modified independently.

Architecture Flow: The user interacts with the React frontend (Presentation Layer), which makes HTTP requests via Axios to the Flask REST API (API Layer). The API layer delegates business logic to specialized Python modules (Business Logic Layer) such as the ML predictor, stock screener, risk analyzer, etc. These modules read/write data from PostgreSQL (Data Layer) and use Redis for caching. Long-running tasks like model training are offloaded to Celery workers (Task Queue Layer) which run asynchronously. A scheduler (APScheduler) triggers the daily data pipeline automatically.

Layer	Technology	Responsibility	Why This Choice
Presentation	React 18 + Vite	UI rendering, interactive charts, user interactions, client-side routing	React's component model enables modular UI; Vite provides 10-100x faster dev builds than Webpack via native ES modules
API Gateway	Flask REST API	60+ endpoints, request validation, authentication, rate limiting, response caching	Flask is lightweight and flexible - ideal for API-only backends. Extension ecosystem (CORS, JWT, Limiter, Caching) adds production features modularly without framework bloat
Real-Time	Flask-SocketIO	WebSocket-based real-time stock quote streaming to the frontend	Enables push-based live price updates without polling; built on top of the existing Flask server
Business Logic	Python Modules (20+)	ML prediction, screening, backtesting, risk analysis, sentiment, fundamental analysis, pattern detection	Python's rich data science ecosystem (PyTorch, pandas, scikit-learn) makes it the natural choice for financial ML
ML Engine	PyTorch	Deep learning model definition, training, inference with GPU acceleration	PyTorch's dynamic computation graph and Pythonic API make it ideal for research-grade model development; CUDA support for GPU training
Task Queue	Celery + Redis	Asynchronous execution of long-running tasks (model training, data pipeline, bulk screening)	Celery is the industry standard for distributed task queues in Python; Redis serves dual purpose as broker and cache
Scheduler	APScheduler	Cron-like job scheduling for automated daily data pipeline at market close	APScheduler integrates natively with Flask; supports cron expressions, interval triggers, and timezone-aware scheduling
Database	PostgreSQL + TimescaleDB	Persistent storage for market data, features, user data, model metrics	PostgreSQL provides ACID compliance and rich SQL; TimescaleDB extension optimizes time-series queries with automatic partitioning and compression
Cache	Redis 7	In-memory caching for market data, predictions, API responses; also serves as Celery message broker	Sub-millisecond read/write latency; supports TTL-based expiration; versatile data structures

Data Flow: External APIs (Yahoo Finance for OHLCV data, NewsAPI/Finnhub for news, Alpha Vantage for additional data) feed raw data into the pipeline. The NSEDataPipeline module fetches and stores raw OHLCV candlestick data into PostgreSQL. The FeatureEngineering module then reads this raw data, computes 150+ technical indicators, and stores the enriched feature set back into a separate database

table. The ML model (MultiTargetStockPredictor) consumes these engineered features to generate predictions. The Flask API serves all computed data, predictions, screening results, risk metrics, and analytics to the React frontend for interactive visualization.

3. Technology Stack - Backend (25+ Libraries)

The backend is written entirely in Python 3.9+ and relies on 25+ libraries. Below is a detailed explanation of each library, why it was chosen, and what specific role it plays in the application.

3.1 Web Framework and Extensions

Library	Purpose	Detailed Explanation
Flask 3.0	REST API Framework	Flask is a micro-framework that provides routing, request/response handling, and a WSGI-compatible server. Unlike Django (which is 'batteries-included'), Flask gives developers full control over which components to use. In Artha Drishti, Flask serves as the API gateway exposing 90+ REST endpoints. Its decorator-based routing (<code>@app.route</code>) makes endpoint definition clean and readable. The app uses the factory pattern via <code>create_app()</code> for configurable initialization.
Flask-CORS	Cross-Origin Resource Sharing	When the React frontend (running on <code>localhost:5173</code>) makes API calls to Flask (running on <code>localhost:5000</code>), browsers block these cross-origin requests by default for security. Flask-CORS adds the necessary <code>Access-Control-Allow-Origin</code> headers to permit these requests. It is configured to allow specific origins (<code>localhost:3000</code> , <code>localhost:5173</code>) rather than wildcard (*) for security.
Flask-JWT-Extended	JWT Authentication	Implements stateless authentication using JSON Web Tokens. When a user logs in (via Google OAuth), the server generates a signed JWT containing the user's identity. This token is sent with every subsequent request in the Authorization header. The server validates the token's signature without needing to store session state. This enables horizontal scaling since any server instance can validate any token. Tokens expire after 24 hours.
Flask-Caching	Response Caching	Caches API responses in memory (SimpleCache backend) or Redis to avoid recomputing expensive results. For example, screening results that don't change frequently are cached with a configurable TTL (Time To Live). When the same request comes again within the TTL, the cached response is returned instantly instead of re-running the screening logic.
Flask-Limiter	API Rate Limiting	Prevents API abuse by limiting the number of requests a client can make within a time window. Configured with sliding window algorithm: 200 requests/day and 50 requests/hour per IP address. Uses in-memory storage in development and Redis in production for distributed rate limiting across multiple server instances.
Flask-SocketIO	WebSocket Support	Enables bidirectional real-time communication between server and client. Used for streaming live stock quotes to the frontend without the overhead of repeated HTTP polling. The server pushes price updates as they become available, providing a responsive real-time experience.

3.2 Machine Learning and Data Science

Library	Purpose	Detailed Explanation
PyTorch >= 2.6	Deep Learning Framework	PyTorch is used to define, train, and run the MultiTargetStockPredictor neural network. It was chosen over TensorFlow because: (1) its dynamic computation graph (eager execution) makes debugging easier, (2) its Pythonic API feels natural, (3) it has excellent GPU support via CUDA for accelerated training, (4) it supports mixed-precision training (AMP) which halves GPU memory usage. The model uses <code>nn.Module</code> for layer definition, <code>autograd</code> for automatic differentiation, <code>DataLoader</code> for batch processing, and <code>ReduceLROnPlateau</code> for learning rate scheduling.

Library	Purpose	Detailed Explanation
scikit-learn	ML Preprocessing and Metrics	Used for: (1) RobustScaler for feature normalization (robust to outliers unlike StandardScaler), (2) train_test_split with time-series awareness, (3) classification metrics (accuracy, precision, recall, F1-score, confusion matrix) for evaluating each prediction head, (4) mutual information scoring for feature importance ranking. It provides the preprocessing pipeline that prepares raw features for neural network consumption.
pandas	DataFrame-Based Data Manipulation	The backbone of all data manipulation in the application. Used for: reading/writing CSV files, SQL query results, time-series resampling, rolling window calculations, merge/join operations, group-by aggregations, and data cleaning (handling NaN/inf values). Every module from FeatureEngineering to RiskAnalytics operates on pandas DataFrames.
NumPy	Numerical Computing	Provides the array computation substrate underlying pandas and PyTorch. Used for: vectorized mathematical operations, statistical calculations (mean, std, percentile), linear algebra (covariance matrices for portfolio risk), random number generation (Monte Carlo simulations), and array broadcasting for efficient batch operations.
SciPy	Scientific Computing	Used for: (1) Kolmogorov-Smirnov test (scipy.stats.ks_2samp) in the drift monitor to detect feature distribution shifts, (2) statistical tests (Jarque-Bera for normality testing in risk analytics), (3) optimization routines for portfolio optimization. Provides the statistical rigor needed for quantitative finance applications.
ta (Technical Analysis)	Technical Indicator Library	Computes 130+ technical indicators from OHLCV data. Includes: RSI, MACD, Bollinger Bands, ADX, Ichimoku Cloud, Stochastic Oscillator, ATR, OBV, CCI, Williams %R, CMF, and many more. Each indicator is implemented as a pandas Series transformation, making it easy to add computed columns to the feature DataFrame.
pandas_ta	Extended Technical Analysis	Supplements the ta library with additional indicators and multi-timeframe support. Provides a more pandas-native API for indicator computation.
transformers (HuggingFace)	Pre-trained NLP Models	Used to load the FinBERT model (ProsusAI/finbert) for financial sentiment analysis. FinBERT is a BERT model fine-tuned on financial text that classifies sentiment as positive/negative/neutral. It provides more accurate financial sentiment than generic NLP tools because it understands financial domain language.

3.3 Data Sources and APIs

Library	Purpose	Detailed Explanation
yfinance	Yahoo Finance Market Data	The primary data source for the entire application. Fetches: (1) daily OHLCV (Open, High, Low, Close, Volume) candlestick data for any stock, (2) fundamental data (P/E ratio, market cap, revenue, etc.), (3) stock news articles, (4) options data. Supports batch downloading of multiple tickers simultaneously. The data pipeline downloads data for the entire NSE universe (~500 stocks) in batches of 50 tickers.
newsapi-python	News API Client	Fetches news articles from 80,000+ news sources worldwide via the NewsAPI service. Used by the SentimentEngine to gather news about specific stocks for sentiment analysis. Returns structured data: title, description, source, URL, published date. Rate-limited to prevent API quota exhaustion.

Library	Purpose	Detailed Explanation
finnhub-python	Finnhub Market Data	Alternative data source providing: company news, market news, earnings calendars, and real-time quotes. Used as both a primary and fallback news source for the sentiment analysis pipeline. Provides more financial-specific news compared to general news APIs.

3.4 Database, Caching, and Task Queue

Library	Purpose	Detailed Explanation
psycopg2-binary	PostgreSQL Database Driver	The most mature and performant PostgreSQL adapter for Python. Provides: connection pooling for efficient reuse of database connections, parameterized queries to prevent SQL injection, bulk insert capabilities via <code>execute_values/execute_many</code> for high-throughput data loading, and COPY protocol support for fastest possible data ingestion. Used by <code>IntegratedPostgreSQL.py</code> for all database operations.
SQLAlchemy	SQL Toolkit / ORM	Provides an optional Object-Relational Mapping layer and a SQL expression language. In Artha Drishti, it is used primarily for: connection management, session handling, and query building. The application uses a hybrid approach - raw SQL via <code>psycopg2</code> for performance-critical bulk operations, and SQLAlchemy for simpler CRUD operations.
redis (Python client)	Redis Cache Client	Python client for the Redis in-memory data store. The <code>RedisClient</code> wrapper class provides: JSON serialization/deserialization for complex Python objects, configurable TTL-based expiration, graceful degradation (falls back silently if Redis is unavailable), and a <code>@redis_cache</code> decorator for transparent function result caching with MD5-hashed keys.
Celery	Distributed Task Queue	Enables asynchronous execution of long-running tasks without blocking the API. When a user requests model training (which may take minutes), the request is placed on a Redis queue and a Celery worker picks it up for background execution. Tasks include: data pipeline runs, feature engineering, model training, bulk screening. Celery Beat provides scheduled periodic tasks (daily pipeline at 4 PM IST, weekly model retraining at 2 AM Saturday).

3.5 Authentication, NLP, and Utilities

Library	Purpose	Detailed Explanation
google-auth	Google OAuth Verification	Verifies Google OAuth2 tokens sent from the frontend. When a user signs in with Google in the React app, the frontend receives a credential JWT from Google. This is sent to the backend, which uses <code>google-auth</code> to verify the token's signature against Google's public keys, extract the user's email/name/avatar, and create or link the account in the local database.
python-dotenv	Environment Variables	Loads configuration from <code>.env</code> files into <code>os.environ</code> . This separates sensitive configuration (database passwords, API keys, JWT secrets) from source code. Different <code>.env</code> files can be used for development, testing, and production environments.
TextBlob	NLP Sentiment (Fallback)	Rule-based NLP library providing basic sentiment analysis (polarity: -1 to +1, subjectivity: 0 to 1). Used as a fallback when FinBERT is not available.
vaderSentiment	Financial Sentiment	VADER (Valence Aware Dictionary and Sentiment Reasoner) is a rule-based sentiment analyzer specifically tuned for social media and financial text. It uses a curated lexicon of sentiment-laden words with intensity ratings.

Library	Purpose	Detailed Explanation
ReportLab	PDF Report Generation	The library generating this very document. Creates PDF files programmatically with full control over layout, typography, tables, and styling. Used by the ExportManager to generate downloadable PDF intelligence reports for stocks.
BeautifulSoup4	HTML Parsing	Parses HTML content from news articles for text extraction. Used by the news scraping pipeline to clean HTML tags and extract plain text content for sentiment analysis.
Gunicorn	Production WSGI Server	A production-grade WSGI HTTP server that replaces Flask's development server. Runs multiple worker processes (configured with 4 workers, 2 threads each) for handling concurrent requests. Essential for production deployment.
APScheduler	Job Scheduler	Advanced Python Scheduler that runs background jobs on cron-like schedules. Configured to trigger the daily data pipeline at 4:00 PM IST on weekdays (Monday-Friday). Supports timezone-aware scheduling, automatic retry with exponential backoff, and NSE holiday calendar awareness to skip non-trading days.

4. Technology Stack - Frontend (14 Libraries)

The frontend is a Single-Page Application (SPA) built with React 18 and Vite. It provides an interactive dark-themed dashboard for visualizing all analytics produced by the backend.

Library	Version	Detailed Explanation of Purpose and Usage
React	18.3	The core UI library using a component-based architecture. Each section of the dashboard (screener, AI analysis, risk, etc.) is a self-contained React component with its own state and lifecycle. React 18 brings: concurrent rendering for smoother UI updates, automatic batching of state updates for better performance, and Suspense for loading states. The app uses functional components exclusively with Hooks (useState for local state, useEffect for side effects like API calls, useCallback for memoized event handlers).
Vite	4.x	The build tool and development server. Vite was chosen over Webpack/Create React App because: (1) it uses native ES modules during development, serving files directly to the browser without bundling - this makes dev server startup nearly instantaneous even for large projects, (2) Hot Module Replacement (HMR) updates are reflected in the browser within milliseconds, (3) production builds use Rollup for efficient code splitting and tree shaking. The vite.config.js configures a proxy to route /api requests to the Flask backend at localhost:5000.
React Router	v7	Handles client-side routing for the SPA. Instead of full page reloads, React Router intercepts navigation and swaps components on the client side. Routes include: / (Dashboard), /screener, /ai-analysis, /portfolio, /backtest, /strategies, /risk, /fundamental, /sentiment, /export, /login. A PrivateRoute wrapper component checks for authentication and redirects unauthenticated users to /login.
Axios	1.7	HTTP client for making API requests to the Flask backend. Chosen over native fetch() because: (1) request interceptors automatically attach the JWT Bearer token to every request, (2) response interceptors catch 401 errors and auto-logout users with expired tokens, (3) automatic JSON parsing, (4) better error handling with structured error objects. All API calls are centralized in a single services/api.js file with functions organized by domain.
Chart.js	4.4	Canvas-based charting library used for rendering OHLCV stock price charts, volume bars, and technical indicator overlays (moving averages, Bollinger Bands). Chart.js was chosen for its performance with large datasets (thousands of data points render smoothly on canvas). The chartjs-adapter-date-fns plugin enables time-series x-axis with proper date formatting and scaling.
react-chartjs-2	5.3	React wrapper components for Chart.js that handle lifecycle management (creating/updating/destroying chart instances when data changes). Provides declarative JSX syntax for chart creation instead of imperative Canvas API calls.
Recharts	2.15	SVG-based React charting library used alongside Chart.js for analytics visualizations (equity curves, risk metric charts, sentiment trends, portfolio allocation pie charts). Recharts excels at responsive, animated charts with React-native tooltip and legend components. SVG-based charts are better for smaller datasets where interactivity (hover effects, tooltips) matters more than raw performance.
Framer Motion	12.4	Declarative animation library for React. Used for: page transition animations, component entrance animations (fade in, slide up), hover effects on cards and buttons, loading state animations, and micro-interactions that make the UI feel alive and responsive. Animations are defined as props (initial, animate, exit) rather than CSS keyframes, making them easier to maintain.
Lucide React	0.474	Modern SVG icon library providing consistent, customizable icons throughout the UI. Used for: navigation menu icons, action button icons, status indicators, chart labels. Each icon is a React component that accepts size, color, and strokeWidth props for consistent styling.

Library	Version	Detailed Explanation of Purpose and Usage
jwt-decode	4.0	Lightweight library for decoding JWT tokens on the client side. When the user logs in and receives a JWT from the backend, jwt-decode extracts the payload (user email, name, expiration time) without needing the server's secret key. Note: this only decodes, it does not verify - verification is done server-side.
@react-oauth/google	0.12	React components and hooks for Google OAuth 2.0 integration. Provides the useGoogleLogin hook that handles the OAuth flow: opens Google's consent screen, receives the credential token, and returns it to the app. The app wraps everything in GoogleOAuthProvider with the Google Client ID.
date-fns	4.1	Lightweight date utility library (alternative to Moment.js). Used for: formatting dates in charts and tables, computing date differences, parsing date strings. Tree-shakeable - only the functions actually used are included in the bundle, unlike Moment.js which imports the entire library.

5. Technology Stack - Infrastructure

Technology	Detailed Explanation of Purpose, Configuration, and Usage
PostgreSQL 14+	The primary relational database storing all persistent data. PostgreSQL was chosen for: (1) ACID compliance ensuring data integrity for financial data, (2) excellent performance with complex queries involving joins, aggregations, and window functions, (3) rich JSON/JSONB support for storing semi-structured data like strategy definitions, (4) UPSERT support (INSERT ON CONFLICT DO UPDATE) for idempotent data loading, (5) compatibility with TimescaleDB. Connection pooling is configured with pool_size=10, max_overflow=20, pool_recycle=3600 seconds.
TimescaleDB	A PostgreSQL extension that transforms PostgreSQL into a purpose-built time-series database. Stock market data is inherently time-series (daily OHLCV rows indexed by date+ticker). TimescaleDB provides: (1) hypertables that automatically partition data by time intervals for faster queries, (2) continuous aggregates for pre-computed rollups, (3) native compression reducing storage by 90%+, (4) time-series specific functions (time_bucket, first, last). The stock_data table is converted to a hypertable partitioned by date.
Redis 7	In-memory key-value store serving two roles: (1) Caching layer - stores frequently accessed data (market prices, screening results, predictions) with TTL-based expiration. Cache reads complete in sub-millisecond time vs. 10-100ms for database queries. The RedisClient class provides JSON serialization, graceful fallback when Redis is down, and a @redis_cache decorator. (2) Message broker for Celery - manages the task queue that distributes work to background workers.
Docker + Docker Compose	Containerization platform packaging the entire application stack into reproducible containers. docker-compose.yml orchestrates 5 services: (1) backend (Flask API on Gunicorn), (2) PostgreSQL/TimescaleDB database, (3) Redis cache/broker, (4) Celery worker for background tasks, (5) Celery Beat for scheduled tasks. All services have health checks, proper dependency ordering, and volume mounts for data persistence across restarts.
Gunicorn	Production-grade WSGI HTTP server replacing Flask's development server. Configured with 4 worker processes and 2 threads each, handling up to 8 concurrent requests. Pre-fork worker model ensures process isolation - if one worker crashes, others continue serving. Binds to 0.0.0.0:5000 in Docker for container networking.
GitHub Actions	CI/CD pipeline for automated testing and deployment. Runs linting (ESLint for frontend, Python syntax checks for backend), unit tests, and can deploy to production on successful builds.

6. Backend Module: application.py - Flask API Server

This is the main entry point of the backend, containing ~5,800 lines of code across 245KB. It is the single largest file in the project and serves as the API gateway that connects the React frontend to all backend functionality.

Initialization: The file uses Flask's app factory pattern via `create_app()`. During initialization, it: (1) creates the Flask app instance, (2) loads configuration from `config.py` and `.env`, (3) initializes CORS with allowed origins, (4) initializes JWT authentication, (5) sets up Flask-Caching with SimpleCache, (6) configures Flask-Limiter for rate limiting, (7) registers a custom JSON provider (NumpySafeJSONProvider) that handles numpy/pandas types (`bool_`, `int64`, `float64`, `ndarray`, `pd.Timestamp`, `Decimal`) that standard JSON serialization cannot handle, (8) imports and instantiates all backend modules (pipeline, predictor, screener, risk analyzer, etc.) with try/except fallbacks for graceful degradation.

Middleware: Two middleware functions run on every request: (1) `@before_request` assigns a UUID request ID for tracing and starts a timer, (2) `@after_request` adds security headers (X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy), records response timing, and logs the request.

Custom Decorators: The file defines reusable decorators: (1) `rate_limit(max_per_minute)` - IP-based sliding window rate limiting, (2) `screen_cache(ttl)` - caches screening results with configurable time-to-live, (3) `@jwt_required()` - protects routes requiring authentication.

Error Handling: Global error handlers catch `BadRequest`, `HTTPException`, and generic `Exception`, returning structured JSON responses with `request_id`, timestamp, error message, and HTTP status code. This ensures the frontend always receives parseable JSON even when errors occur.

6.1 API Endpoint Groups (90+ Endpoints)

Group	Count	Example Endpoints	Detailed Description
Health/Status	5	<code>/api/health</code> , <code>/api/health/live</code> , <code>/api/health/ready</code> , <code>/api/health/model</code> , <code>/api/stats</code>	Health check endpoints for monitoring. <code>/health</code> returns basic OK status. <code>/health/live</code> checks if the server is responsive (liveness probe for Kubernetes). <code>/health/ready</code> checks if all dependencies (database, Redis, models) are available (readiness probe). <code>/health/model</code> reports ML model status. <code>/api/stats</code> returns system statistics.
Authentication	4	<code>/api/auth/google</code> , <code>/api/auth/login</code> , <code>/api/auth/register</code> , <code>/api/auth/refresh</code>	User authentication endpoints. <code>/auth/google</code> receives the Google OAuth credential token, verifies it, creates/links the user account, and returns a JWT session token. <code>/auth/login</code> and <code>/auth/register</code> provide local authentication. <code>/auth/refresh</code> generates a new JWT before the current one expires.
Market Data	8	<code>/api/stocks</code> , <code>/api/history/{ticker}</code> , <code>/api/quote/{ticker}</code> , <code>/api/quotes/batch</code> , <code>/api/market/movers</code> , <code>/api/market/overview</code> , <code>/api/stocks/search</code>	Stock data endpoints. <code>/stocks</code> lists all available stocks. <code>/history/{ticker}</code> returns full OHLCV history. <code>/quote/{ticker}</code> returns the latest price. <code>/quotes/batch</code> returns prices for multiple tickers in one call. <code>/market/movers</code> returns top gainers/losers. <code>/market/overview</code> returns market-wide statistics.

Group	Count	Example Endpoints	Detailed Description
Screening	12+	/api/screen/momentum, /api/screen/piotroski, /api/screen/swing, /api/screen/breakout, /api/screen/value, /api/screen/garp, /api/screen/custom/run/{name}	Stock screening endpoints - one per strategy. Each endpoint runs the respective screening strategy against the stock universe, computing technical indicators and evaluating strategy-specific conditions. Returns stocks that pass with confidence scores and signal metadata. Custom strategy endpoint runs user-defined strategies.
ML Prediction	6	/api/predict/{ticker}, /api/price-target/{ticker}, /api/price-target/{ticker}/levels, /api/price-targets/batch, /api/train/{ticker}, /api/model/info	AI prediction endpoints. /predict/{ticker} runs the neural network to get 5-target predictions. /price-target/{ticker} returns the predicted price target with support/resistance levels. /price-targets/batch handles multiple tickers. /train/{ticker} triggers model training. /model/info returns model architecture and performance details.
Backtesting	5	/api/backtest/{strategy}, /api/backtest/compare, /api/backtest/walk-forward, /api/backtest/regimes/{ticker}, /api/backtest/custom/{name}	Strategy backtesting endpoints. /backtest/{strategy} simulates a strategy on historical data and returns performance metrics, equity curve, and trade log. /backtest/compare evaluates multiple strategies side-by-side. /backtest/walk-forward runs walk-forward cross-validation. /backtest/regimes/{ticker} detects market regimes.
Portfolio	5	/api/portfolio/watchlist, /api/portfolio/holdings, /api/portfolio/summary, /api/portfolio/add, /api/portfolio/remove	Portfolio management endpoints for adding/removing watchlist stocks, managing holdings, and computing portfolio summary with current market values and P&L.;
Risk Analytics	4	/api/risk/{ticker}, /api/risk/portfolio, /api/risk/compare, /api/risk/frontier	Risk analysis endpoints. /risk/{ticker} computes 30+ risk metrics for a single stock. /risk/portfolio computes portfolio-level risk including Component VaR. /risk/frontier generates Monte Carlo efficient frontier.
Fundamental	4	/api/fundamental/{ticker}, /api/fundamental/piotroski/{ticker}, /api/fundamental/compare, /api/fundamental/score/{ticker}	Fundamental analysis endpoints. Returns Piotroski F-Score with sub-score breakdown, proprietary composite score (0-100), and multi-stock comparison across all fundamental dimensions.
Sentiment	3	/api/sentiment/{ticker}, /api/sentiment/history, /api/sentiment/sources	Sentiment analysis endpoints. Fetches news, runs FinBERT/VADER/TextBlob sentiment analysis, returns aggregated sentiment scores, trend over time, and source-level breakdown.
Export	4	/api/export/report, /api/export/csv, /api/export/json, /api/export/html	Report export endpoints. Generates downloadable reports in multiple formats. The HTML report is a self-contained intelligence dossier with dark theme styling, combining AI predictions, fundamental scores, risk metrics, and technical analysis.
Scheduler	3	/api/scheduler/status, /api/scheduler/trigger, /api/scheduler/schedule	Scheduler control endpoints. /status returns whether the scheduler is running and next execution time. /trigger manually runs the pipeline immediately. /schedule allows changing the schedule time.
Data Quality	4	/api/data-quality/report, /api/data-quality/reconcile, /api/stocks/gaps/{ticker}, /api/features/quality-report	Data quality monitoring endpoints for checking data completeness, detecting gaps in time series, reconciling data sources, and validating computed features.

7. Backend Module: MLPredictor.py - Deep Learning Engine

This is the crown jewel of the application at 712KB (~13,700 lines). It contains the complete end-to-end deep learning prediction system including model architecture, training pipeline, inference engine, production safety guards, and model lifecycle management.

7.1 Neural Network Architecture (MultiTargetStockModel)

The neural network processes a sequence of 40 time steps, where each time step contains 145+ engineered features. The architecture processes this input through multiple stages, each building on the previous:

Layer #	Layer Name	Technical Details	Purpose and Intuition
1	BatchNorm1d	Normalizes each feature across the batch to zero mean and unit variance	Stabilizes training by ensuring all features are on the same scale. Without this, features with large values (e.g., volume in millions) would dominate features with small values (e.g., RSI 0-100). Batch normalization also acts as a regularizer.
2	Sinusoidal Positional Encoding	$PE(pos, 2i) = \sin(pos/10000^{(2i/d)})$, $PE(pos, 2i+1) = \cos(pos/10000^{(2i/d)})$	LSTMs and convolutions process sequences, but they don't inherently know the position of each element. Positional encoding adds position information by injecting unique sine/cosine patterns at each time step. This technique comes from the Transformer architecture (Vaswani et al., 2017). The sinusoidal form was chosen because it allows the model to attend to relative positions.
3	Multi-Scale Temporal Convolution (TCN)	4 parallel Conv1d branches with dilation rates {1, 2, 4, 8}, followed by concatenation and linear projection	Each branch captures patterns at a different temporal granularity. Dilation rate 1 sees 1-day patterns (daily noise), rate 2 sees 2-day patterns, rate 4 sees weekly patterns, and rate 8 sees bi-weekly patterns. By running all branches in parallel and concatenating outputs, the model captures both short-term and long-term temporal patterns simultaneously. Dilated convolutions have receptive fields that grow exponentially with depth while keeping the parameter count linear.
4	Bidirectional LSTM (2 layers)	Forward LSTM processes day 1->40, backward LSTM processes day 40->1; outputs are concatenated	LSTM (Long Short-Term Memory) is a recurrent neural network variant designed to handle long-range dependencies in sequential data. It uses three gates (forget, input, output) to control information flow, solving the vanishing gradient problem that plagues vanilla RNNs. Bidirectional processing lets the model see both past context (forward) and future context (backward) when encoding each time step. Two stacked layers allow the model to learn hierarchical temporal features.
5	Spatial Dropout	Randomly drops entire feature channels during training (dropout rate specified in config)	Unlike standard dropout which drops individual neurons, spatial dropout drops entire feature dimensions across all time steps. This forces the model to not rely too heavily on any single feature, improving generalization. Essential for preventing overfitting on 145+ features.

Layer #	Layer Name	Technical Details	Purpose and Intuition
6	Multi-Head Self-Attention	Learned Query (Q), Key (K), Value (V) projections; $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$; 8 attention heads	Self-attention computes a weighted sum of all time steps, where the weights are learned based on content similarity. This allows the model to focus on the most informative days in the 40-day window. For example, a day with an earnings announcement or a significant price move might receive higher attention weight. Multiple heads (8) allow the model to attend to different aspects simultaneously (one head might focus on volume patterns, another on price momentum).
7	Feed-Forward Network (FFN)	Two linear layers with GELU activation and residual connection	Adds non-linear transformation capacity after attention. The residual connection (adding input to output) helps gradient flow during training and allows the model to learn incremental refinements.
8	Temporal Attention Pooling	Learned attention weights over time steps to produce a single summary vector	Collapses the 40-step sequence into a single fixed-length vector by computing a weighted average across time steps. The weights are learned, so the model decides which days matter most for prediction. This is more expressive than simple averaging or taking the last time step.
9	5 Decoder Heads	Independent Linear(64->32)->GELU->Linear(32->output) for each target	Each prediction target has its own dedicated decoder head with separate parameters. This allows each head to specialize: the direction head might focus on momentum features, while the volatility head focuses on ATR and Bollinger Band features. The heads share the encoder but have independent decoders, enabling multi-task learning.
10	Confidence Gating	Learned weighted combination of per-head softmax confidences	The model self-assesses its prediction reliability by computing a confidence score. This is a weighted average of the maximum softmax probabilities from each classification head. High confidence (>0.65) suggests the model is sure; low confidence (<0.35) suggests uncertainty. This is critical for risk management - positions should be sized proportionally to model confidence.

7.2 Five Prediction Targets

The model simultaneously predicts five interdependent financial targets. This multi-target approach provides a complete picture of a stock's outlook:

Target	Type	Output	Detailed Explanation
Direction	3-Class Classification	Bullish / Bearish / Neutral	The primary prediction target. Predicts whether the stock price will go up (Bullish), down (Bearish), or remain flat (Neutral) over the prediction horizon. This is the main driver during training - other heads are gradient-isolated to prevent them from interfering with direction learning.
Volatility Regime	3-Class Classification	Low / Medium / High	Predicts the expected volatility level. Low volatility regimes favor trend-following strategies; high volatility regimes require wider stop-losses and smaller position sizes. This helps investors adjust their risk management based on expected market conditions.

Target	Type	Output	Detailed Explanation
Risk Level	3-Class Classification	Low / Medium / High	Combines multiple risk factors (volatility, drawdown potential, liquidity) into a single risk assessment. A stock might be directionally bullish but high-risk (e.g., a small-cap stock with thin trading volume). This target helps risk-averse investors filter opportunities.
Trend Strength	Regression (0.0 - 1.0)	Continuous value	Quantifies how strong the current trend is. A value near 1.0 indicates a strong, persistent trend; near 0.0 indicates a weak or non-existent trend. This is useful for position sizing - stronger trends justify larger positions.
Optimal Action	5-Class Classification	Strong Buy / Buy / Hold / Sell / Strong Sell	The most actionable prediction. Combines direction, volatility, and risk into a single recommended action with five granularity levels. Asymmetric confidence gating is applied: BUY signals require >0.65 confidence, SELL signals require <0.35 confidence. This conservative approach reduces false signals.

7.3 Training Pipeline

The training pipeline implements multiple advanced techniques:

- **Multi-Task Loss Function:** $L_{total} = \sum(\lambda_i * L_i)$ where λ_i are learnable task weights and L_i are individual task losses. Classification heads use Focal Loss ($\gamma=2.0$) which down-weights easy examples and focuses on hard-to-classify stocks, addressing class imbalance. The regression head uses MSE loss.
- **6-Axis Regularization:** (1) Input noise injection, (2) Feature dropout at 28%, (3) Temporal cutout at 20% (randomly zeroing out time windows), (4) Spatial dropout (dropping entire feature channels), (5) Mixup augmentation (interpolating between training examples), (6) Focal loss (implicit regularization by focusing on hard examples). This comprehensive regularization prevents overfitting on historical data.
- **Adversarial Training (FGSM):** Fast Gradient Sign Method generates adversarial examples by adding small perturbations in the direction that maximally increases the loss. Training on these adversarial examples improves model robustness to input noise.
- **R-Drop Consistency Regularization:** Runs the same input through the model twice with different dropout masks and penalizes inconsistency between the two outputs. This improves prediction stability.
- **EMA (Exponential Moving Average):** Maintains a smoothed copy of model weights updated as: $\theta_{ema} = \alpha * \theta_{ema} + (1-\alpha) * \theta_{model}$. The EMA model is used for inference as it produces more stable predictions than the raw training model.
- **PCGrad (Projecting Conflicting Gradients):** When gradients from different task heads conflict (point in opposite directions), PCGrad projects one gradient onto the normal plane of the other, preventing tasks from fighting each other during optimization.
- **Time-Series Aware Splitting:** Training data is split chronologically (not randomly) to prevent data leakage. The model only trains on past data and validates on future data, simulating real trading conditions.
- **Adam Optimizer with ReduceLROnPlateau:** Learning rate is automatically reduced when validation loss plateaus. Early stopping halts training when no improvement is seen for a configurable number of epochs (patience).

7.4 Production Safety and Monitoring

The UnifiedStockPredictor wrapper class provides production-grade infrastructure:

- **ModelDegradationCircuitBreaker:** Automatically halts predictions if model quality degrades below acceptable thresholds. Monitors rolling accuracy and triggers alerts when performance drops.
- **ProductionModelRegistry:** Versions every trained model with metadata (training date, metrics, feature set). Enables rollback to previous versions if a new model underperforms.

- **ProductionSafetyGuard:** Checks for data staleness (old data), regime anomalies (unusual market conditions), and corporate actions (stock splits, dividends) that might invalidate predictions.
- **DynamicKellyCalculator:** Computes position sizes using the Kelly Criterion ($f^* = (bp - q) / b$ where b = odds, p = win probability, q = loss probability). Uses a fractional Kelly (25%) for more conservative sizing.
- **PredictionRecorder:** Stores every prediction in PostgreSQL with timestamp, creating an audit trail. PredictionTracker later verifies predictions against actual outcomes to compute real accuracy.
- **DriftMonitor:** Uses Population Stability Index (PSI) to detect when feature distributions shift from the training baseline. $PSI < 0.10$ = stable; $0.10-0.25$ = moderate drift (monitor closely); ≥ 0.25 = severe drift (retrain recommended).
- **TemperatureScaling:** Calibrates prediction confidence scores so that when the model says 80% confident, it is actually correct 80% of the time. Monitors Expected Calibration Error (ECE) and Maximum Calibration Error (MCE).

8. Backend Module: StockScreener.py - Multi-Strategy Screening Engine

The screening engine (196KB, ~4,600 lines) evaluates every stock against 18+ independent quantitative strategies. The key innovation is the Committee-of-Experts approach: instead of relying on a single strategy (which introduces bias), the system requires consensus across multiple fundamentally different strategies.

8.1 Strategy Registry

Each strategy is implemented as an independent evaluator with its own rule set. Strategies are organized into 7 categories:

Category	Strategy Name	How It Works (Detailed Rule Set)
Trend Following	Trend Rider	Identifies stocks in strong uptrends. Conditions: price above 50-day and 200-day SMA, 50-day SMA above 200-day SMA (Golden Cross alignment), ADX > 25 (strong trend), +DI > -DI (bullish directional movement). Confidence is proportional to ADX strength.
Trend Following	Golden Cross	Detects the classic Golden Cross pattern where the 50-day SMA crosses above the 200-day SMA. This is a widely followed bullish signal indicating a shift from medium-term downtrend to uptrend. Requires the crossover to have occurred within the last 5 trading days.
Trend Following	Ichimoku Cloud Breakout	Uses the Ichimoku Cloud system (5 lines: Tenkan-sen, Kijun-sen, Senkou Span A/B, Chikou Span). Triggers when price breaks above the cloud, Tenkan-sen is above Kijun-sen, and the Chikou Span confirms by being above the cloud. This provides a holistic view of support, resistance, and trend direction.
Trend Following	Supertrend Bullish	Uses the Supertrend indicator (period=10, multiplier=3x ATR). Triggers when price crosses above the Supertrend line. The Supertrend acts as a dynamic trailing stop-loss that adapts to volatility. Bullish when price is above the line.
Momentum	Momentum Hunter	Identifies stocks with accelerating upward momentum. Conditions: RSI between 50-70 (strong but not overbought), MACD line above signal line, MACD histogram increasing (accelerating momentum), price above 20-day EMA, volume above 20-day average (confirming participation). Confidence based on RSI distance from 50.
Momentum	Breakout Scanner	Detects stocks breaking out of consolidation ranges. Conditions: price breaking above the upper Bollinger Band (2 standard deviations), volume surge > 1.5x the 20-day average (confirming the breakout), ADX > 20 (directional movement). High-volume breakouts above resistance are among the most reliable entry signals.
Momentum	52-Week High Momentum	Finds stocks near their 52-week high. Stocks making new highs tend to continue making new highs (momentum effect). Conditions: price within 5% of 52-week high, RSI > 50 but < 80 (not extremely overbought), positive 3-month return.
Swing Trading	Swing Trader	Identifies short-term reversal opportunities. Conditions: RSI < 35 (oversold), price near lower Bollinger Band (oversold on volatility basis), MACD histogram showing bullish divergence (price making new lows but MACD making higher lows). Targets 5-10 day holding periods.
Value	Value Bounce	Finds undervalued stocks showing signs of recovery. Conditions: P/E ratio below sector median, price near 52-week low but bouncing (higher low formed), positive money flow (accumulation by smart money), RSI turning up from oversold territory.

Category	Strategy Name	How It Works (Detailed Rule Set)
Value	GARP (Growth at Reasonable Price)	Combines value and growth screening. Conditions: PEG ratio < 1.5 (earnings growth at a reasonable price), P/E below industry average, positive earnings growth, positive revenue growth. This strategy was popularized by Peter Lynch.
Mean Reversion	RSI Divergence	Detects bullish RSI divergence: price makes a lower low but RSI makes a higher low. This divergence suggests selling pressure is weakening and a reversal is likely. The strategy computes divergence over 14-day RSI with a lookback period of 5-20 bars.
Mean Reversion	Bollinger Squeeze	Identifies low-volatility compression periods (Bollinger Band width at multi-week lows) followed by expansion. When Bollinger Bands squeeze tightly, it indicates a period of consolidation that often precedes a significant directional move. Triggers when the band width starts expanding from a squeeze.
Mean Reversion	Contrarian Recovery	Finds stocks that have experienced significant recent declines but show signs of bottoming. Conditions: 1-month return < -15%, current RSI < 30, volume declining on drops (selling exhaustion), and any bullish candlestick pattern near the low (hammer, engulfing).
Pattern	Double Bottom	Detects the double bottom reversal pattern: two consecutive troughs at approximately the same price level, with a peak (neckline) between them. Triggers when price breaks above the neckline with confirming volume. The price target equals the neckline price plus the pattern height.
Multi-Factor	Volume Surge	Identifies unusual volume activity indicating institutional interest. Conditions: current volume > 2x the 20-day average, price change > 0 (bullish volume), OBV (On-Balance Volume) trending up. Volume surges often precede significant price moves as institutions accumulate positions.
Multi-Factor	Smart Money	Combines multiple signals that institutional investors typically look for: positive money flow index (MFI > 50), increasing OBV, price above VWAP, positive Chaikin Money Flow. When multiple accumulation indicators agree, it suggests institutional buying.
Multi-Factor	Triple MACD Alignment	Requires MACD signals across three timeframes (daily, weekly, monthly) to all be bullish simultaneously. MACD line above signal line on all three timeframes indicates alignment across short-term, medium-term, and long-term momentum.

8.2 Committee Consensus Mechanism

After all strategies evaluate a stock independently, the committee consensus mechanism aggregates the results. The process works as follows:

- **Step 1 - Universal Indicator Computation:** Before running any strategy, the system computes a universal set of 20+ technical indicators (RSI, MACD, Bollinger Bands, ADX, Ichimoku, Supertrend, ATR, OBV, Stochastic, ROC, Williams %R, CMF, VWAP proxy, etc.) once per stock. This shared indicator set is passed to all strategies, eliminating redundant computation and ensuring consistency.
- **Step 2 - Independent Evaluation:** Each strategy independently evaluates its conditions against the shared indicators. Each strategy produces: Pass/Fail boolean, confidence score (0-100), signal type (Bullish/Bearish/Neutral), and detailed metadata about which conditions were met and which failed.
- **Step 3 - Counting Votes:** The system counts how many strategies passed (N_pass) and computes the average confidence across passing strategies (C_avg).
- **Step 4 - Consensus Threshold:** A stock qualifies for recommendation if and only if: N_pass >= T_min (default: 2 strategies must agree) AND C_avg >= C_min (default: 60% average confidence). Both thresholds are user-configurable.
- **Step 5 - Verdict Classification:** Based on the ratio N_pass/N_total, the stock receives a verdict: Strong Consensus (>70% of strategies agree), Moderate Consensus (40-70%), Weak Consensus (20-40%), or No Consensus (<20%).

- **Step 6 - Custom Strategy Support:** Users can define custom strategies as JSON rule sets specifying indicator conditions. These are dynamically evaluated at runtime without requiring a restart or redeployment.

9. Backend Module: FeatureEngineering.py - Technical Indicator Pipeline

This module (113KB) is responsible for transforming raw OHLCV data into 150+ engineered features that serve as input to the ML model. Feature engineering is often the most important factor in model performance - the quality of features determines the ceiling of what any model can learn.

Category	Co unt	Features	Why These Features Matter
Price-Based	15+	Returns, log returns, price ratios (close/open, high/low), gap percentage, candle body/shadow ratios	Raw price transformations capture the basic dynamics of price movement. Log returns are preferred over simple returns because they are additive across time and more closely follow a normal distribution. Body/shadow ratios encode candlestick patterns numerically.
Moving Averages	12+	SMA(5,10,20,50,100,200), EMA(9,21,50), WMA, DEMA, TEMA, KAMA	Moving averages smooth out noise and reveal underlying trends. Short-term MAs (5,10) capture momentum; long-term MAs (100,200) capture macro trends. Crossovers between different MAs generate trading signals. EMA gives more weight to recent prices. KAMA (Kaufman Adaptive) automatically adjusts its smoothing based on market noise.
Momentum Oscillators	10+	RSI(14), MACD(12,26,9), Stochastic(14,3), Williams %R, ROC, CCI, MFI	Oscillators measure the speed and magnitude of price changes. RSI identifies overbought (>70) and oversold (<30) conditions. MACD measures momentum by comparing two EMAs. Stochastic shows where the close is relative to the recent high-low range. These features help the model detect potential reversals.
Volatility	8+	Bollinger Bands(20,2), ATR(14), Keltner Channels, Donchian Channels, historical volatility	Volatility features capture the degree of price uncertainty. Bollinger Band width indicates market volatility regime. ATR measures average true range for position sizing and stop-loss placement. High/low volatility periods have different statistical properties that the model learns to distinguish.
Volume	6+	OBV, VWAP proxy, Accumulation/Distribution, CMF, Force Index, Ease of Movement	Volume confirms or contradicts price movements. OBV (On-Balance Volume) shows whether volume flows with or against the trend. CMF (Chaikin Money Flow) measures buying/selling pressure. Price moves on high volume are more significant than those on low volume.
Trend	8+	ADX(14), Ichimoku Cloud (5 lines), Supertrend, Aroon, Parabolic SAR, TRIX	Trend indicators determine trend direction and strength. ADX quantifies trend strength regardless of direction. Ichimoku provides a complete picture of support/resistance/trend in one indicator. The model uses these to distinguish between trending and mean-reverting market regimes.
Statistical	15+	Rolling mean/std/skew/kurtosis, Z-scores, percentile ranks, regime detection	Statistical features capture distribution properties of returns. Skewness measures tail asymmetry (negative skew = crash risk). Kurtosis measures tail fatness (high kurtosis = more extreme moves). Z-scores normalize features relative to their recent history.
Multi-Timeframe	20+	5-day, 10-day, 20-day versions of key indicators	Different timeframes capture different trader horizons. Day traders look at 5-day indicators; swing traders at 10-20 day; position traders at 50-200 day. Including multiple timeframes helps the model understand the full market context.

Category	Co unt	Features	Why These Features Matter
Interaction	10+	RSI x Volume, MACD x ADX, cross-feature products	Interaction features capture non-linear relationships between indicators. For example, a bullish MACD signal is more meaningful when ADX is high (strong trend) than when ADX is low (choppy market). These products help the model learn conditional relationships.
Target Labels	5	Direction (3-class), volatility regime, risk level, trend strength, optimal action	Target engineering transforms future price data into the labels the model learns to predict. Direction labels use configurable thresholds to classify future returns as bullish/bearish/neutral. Volatility regimes are assigned based on percentile ranking of historical volatility.

Pipeline Execution: The feature engineering pipeline runs daily: (1) Fetch raw OHLCV data from PostgreSQL, (2) Compute all 150+ features using the ta library and custom calculations, (3) Clean NaN/inf values (forward fill, then backward fill, then drop remaining), (4) Store the enriched feature DataFrame back to the features_engineered table in PostgreSQL. This runs automatically via the scheduler at market close.

Advanced Features (AdvancedFeatureEngine.py): An extended feature engine adds: wavelet features (multi-resolution analysis via discrete wavelet transform), Fourier features (frequency-domain analysis), entropy features (Shannon entropy, approximate entropy for regime detection), fractal features (fractal dimension, lacunarity), cross-asset features (correlation with NIFTY 50, India VIX, USD/INR, Brent Crude), and microstructure features (bid-ask spread proxies). These capture market dynamics invisible to traditional indicators.

10. Backend Module: IntegratedPostgreSQL.py - Data Pipeline

This module (105KB) manages the complete data lifecycle: fetching stock data from external sources, storing it in PostgreSQL, and maintaining the stock universe.

NSEDataPipeline Class: The central class handles:

- **Stock Universe Management:** Maintains the list of NSE-listed stocks by fetching the NSE equity list from `archives.nseindia.com`. Covers NIFTY 50, NIFTY Next 50, and NIFTY 500 constituents (~500 stocks). Each stock is stored with metadata: symbol, name, sector, industry, exchange.
- **Data Fetching:** Uses `yfinance` to download daily OHLCV data. Downloads in batches of 50 tickers to avoid API rate limits. Supports both full historical downloads and incremental updates (only fetching data since the last available date). Handles failures gracefully with retry logic.
- **Data Storage (Upsert Pattern):** Uses PostgreSQL's `INSERT ON CONFLICT DO UPDATE` for idempotent data loading. This means running the pipeline twice with the same data doesn't create duplicates - existing rows are updated, new rows are inserted. Implementation uses a temporary table strategy: (1) create temp table, (2) COPY data into temp table (fastest PostgreSQL bulk load), (3) `INSERT INTO main table FROM temp table ON CONFLICT DO UPDATE`.
- **Connection Management:** Uses `psycopg2` with connection pooling for efficient database connection reuse. Connections are returned to the pool after each operation rather than being closed and reopened.
- **TimescaleDB Integration:** The `stock_data` table is converted to a TimescaleDB hypertable partitioned by the date column. This enables: automatic time-based partitioning, chunk-level operations (drop old data by dropping chunks), and optimized time-range queries.

11. Backend Module: BacktestEngine.py - Backtesting System

The backtesting engine (37KB) simulates strategy execution on historical data to evaluate how a strategy would have performed in the past. This is essential for validating strategies before deploying real capital.

Key Components:

- **DataNormalizer:** Adjusts raw price data for corporate actions (stock splits, dividends) to ensure backtesting accuracy. Also adds derived metrics: ATR for stop-loss calculation, historical volatility, volume moving averages, and anomaly detection flags.
- **RegimeDetector:** Classifies each period into market regimes: BULL (sustained uptrend), BEAR (sustained downtrend), HIGH_VOL (volatile/uncertain), LOW_VOL (calm/trending), CHOPPY (range-bound). Regime detection uses a combination of moving average slopes, volatility percentiles, and trend strength indicators.
- **Realistic Execution Simulation:** Unlike simple backtests that assume instant fills at exact prices, this engine models: (a) Volume-aware slippage: $\text{slippage} = \text{base\%} \times (1 + \text{order_size}/\text{avg_volume} \times \text{impact}) \times \text{atr_ratio}$. Larger orders in illiquid stocks experience more slippage. (b) Gap handling: if the market gaps past a stop-loss, the exit price is the open price, not the stop price. (c) Open-price fills: orders are filled at the next day's open, not the current close. (d) Transaction costs: configurable commission per trade.
- **Position Sizing Modes:** (1) Risk-based: $\text{size} = \text{risk_amount} / (\text{entry} - \text{stop})$, (2) Volatility-adjusted: size inversely proportional to ATR, (3) Fixed-dollar: fixed amount per trade, (4) Portfolio-risk: limits total portfolio risk across all open positions.
- **ATR-Based Risk Management:** Stop-losses are set at $\text{entry_price} - N \times \text{ATR}$ (configurable N, typically 2-3). Take-profit targets are set at $\text{entry_price} + M \times \text{ATR}$ (typically 3-5). ATR adapts to current volatility, so stops are wider in volatile markets and tighter in calm markets.
- **Performance Metrics:** Total return, annualized return, Sharpe ratio, Sortino ratio, max drawdown, win rate, profit factor, average trade duration, number of trades, expectancy per trade. All metrics compared against a buy-and-hold benchmark.
- **Walk-Forward Validation (walkforward.py):** PurgedWalkForwardSplitter with `train_days=504` (~2 years), `test_days=63` (~1 quarter), `embargo_days=10` (gap between train and test to prevent label leakage). The embargo period ensures no overlap between the training target labels and test evaluation period. Walk-forward prevents overfitting by ensuring the strategy is always tested on unseen future data.

12. Backend Module: RiskAnalytics.py - Quantitative Risk Analysis

The risk analytics module (23KB) computes 30+ risk metrics organized into 7 categories, providing a comprehensive risk profile for individual stocks and entire portfolios.

Category	Metrics	Detailed Explanation
Volatility	Annualized volatility, downside deviation, upside volatility, volatility skew ratio	Annualized volatility = daily std * sqrt(252). Downside deviation only considers negative returns (Sortino denominator). Volatility skew ratio = downside vol / upside vol; a ratio > 1 means the stock falls harder than it rises.
Drawdown	Max drawdown, average drawdown, drawdown duration, recovery factor	Drawdown = decline from peak to trough. Max drawdown is the worst-case decline. Recovery factor = total return / max drawdown (how well the strategy recovers). Drawdown duration measures how long it takes to recover to the previous peak.
Value at Risk	Parametric VaR (95%, 99%), Historical VaR, CVaR (Expected Shortfall)	VaR answers: what is the maximum loss at a given confidence level? 95% VaR = -2.6% means there is a 5% chance of losing more than 2.6% in one day. CVaR (Conditional VaR / Expected Shortfall) answers: when VaR is breached, what is the expected loss? CVaR is always worse than VaR and better captures tail risk.
Tail Risk	Skewness, excess kurtosis, Jarque-Bera statistic, tail ratio	Skewness measures asymmetry: negative skew = more frequent small gains but rare large losses (crash risk). Kurtosis measures tail fatness: high kurtosis = more extreme events than a normal distribution predicts. Jarque-Bera tests if returns are normally distributed (they usually are not).
Risk-Adjusted Returns	Sharpe ratio, Sortino ratio, Calmar ratio, Omega ratio, Information ratio	Sharpe = (return - risk_free) / volatility. Sortino = (return - risk_free) / downside_deviation (only penalizes downside risk). Calmar = annualized_return / max_drawdown. Omega = probability-weighted gains / probability-weighted losses. Information ratio = excess_return / tracking_error.
Market Risk	Beta, Alpha, Treynor ratio, R-squared, tracking error	Beta measures systematic risk (sensitivity to market movements). Alpha = actual_return - expected_return (skill-based return). Treynor = excess_return / beta (return per unit of systematic risk). R-squared = how much of the stock's variance is explained by the market.
Advanced	Hurst exponent, Ulcer index, pain ratio, gain-loss ratio	Hurst exponent (via Rescaled Range analysis): $H < 0.5$ = mean-reverting, $H = 0.5$ = random walk, $H > 0.5$ = trending. Ulcer index measures investor anxiety from drawdowns. Pain ratio = excess_return / ulcer_index.

Portfolio-Level Analysis:

- **Component VaR Decomposition:** $CVaR_i = w_i * \beta_i^{\text{portfolio}} * VaR_{\text{portfolio}}$. Decomposes total portfolio risk to each asset's contribution. This reveals which positions are the biggest risk contributors, enabling targeted risk reduction.
- **Diversification Ratio:** $DR = \sum(w_i * \sigma_i) / \sigma_{\text{portfolio}}$. If $DR > 1$, the portfolio benefits from diversification (imperfect correlations reduce overall risk). Higher DR = better diversification.
- **Correlation Matrix:** Computes pairwise return correlations between all portfolio assets. Low correlations between assets provide diversification benefits.
- **Monte Carlo Efficient Frontier:** Generates N random portfolios using Dirichlet-distributed random weights ($w \sim \text{Dir}(\alpha=1)$ ensures uniform sampling on the weight simplex). For each portfolio, computes expected return and volatility. Identifies: (1) Maximum Sharpe Ratio portfolio (optimal risk-adjusted return), (2) Minimum Volatility portfolio (lowest risk). Returns frontier coordinates for visualization.

13. Backend Module: SentimentEngine.py - Multi-Source Sentiment Analysis

The sentiment engine (30KB) aggregates news from multiple sources and computes sentiment scores using an ensemble of three NLP analyzers:

- **FinBERT (Primary):** A BERT model fine-tuned on 50,000+ financial texts (ProsusAI/finbert via HuggingFace Transformers). BERT (Bidirectional Encoder Representations from Transformers) uses self-attention to understand context from both directions. FinBERT specifically understands financial language - it knows that 'the stock tanked' is negative and 'beat earnings estimates' is positive. Lazy-loaded singleton to avoid loading the 440MB model until first use.
 - **VADER (Secondary):** Valence Aware Dictionary and Sentiment Reasoner. A rule-based analyzer with a curated lexicon of sentiment-laden words, each with intensity ratings. Handles: negations ('not good' = negative), degree modifiers ('very good' = more positive), punctuation emphasis, capitalization, emoji. Fast and requires no GPU.
 - **TextBlob (Fallback):** Simple NLP library providing polarity (-1 to +1) and subjectivity (0 to 1) scores. Uses a Naive Bayes classifier trained on movie reviews. Less accurate for financial text but serves as a reliable fallback.
 - **Temporal Decay Weighting:** Recent news articles receive exponentially higher weights than older articles. A news article from today has more impact than one from two weeks ago. Decay formula: $\text{weight} = \exp(-\text{decay_rate} * \text{age_in_days})$.
 - **Event Classification:** Articles are classified into event types (earnings, M&A, regulatory, product launch) with domain-specific sentiment multipliers. An earnings beat amplifies positive sentiment more than a generic positive article.
 - **News Sources:** Finnhub API (primary - financial-specific news), yfinance Ticker.news (free fallback), Alpha Vantage (additional source). Results are cached for 10 minutes with rate limiting to prevent API quota exhaustion.
-

14. Backend Module: FundamentalAnalysis.py - Fundamental Scoring

This module (17KB) implements two complementary fundamental scoring frameworks:

Piotroski F-Score (0-9 points): Developed by Professor Joseph Piotroski (Stanford), this academic scoring system uses 9 binary signals based on financial statements. Each criterion is worth 1 point:

- **Profitability (4 points):** (1) ROA > 0 (profitable), (2) Operating cash flow > 0 (generates cash), (3) ROA increasing year-over-year (improving profitability), (4) Operating cash flow > net income (earnings quality - cash flow backs up reported profits)
- **Leverage/Liquidity (3 points):** (5) Long-term debt ratio decreasing (deleveraging), (6) Current ratio increasing (improving short-term liquidity), (7) No new share issuance (no dilution of existing shareholders)
- **Operating Efficiency (2 points):** (8) Gross margin increasing (improving pricing power or cost efficiency), (9) Asset turnover increasing (more revenue per dollar of assets)
- Scores 8-9: Strong Buy, 7: Buy, 5-6: Hold, 3-4: Sell, 0-2: Strong Sell

Proprietary 5-Category Composite Score (0-100):

Category	Weight	Metrics Evaluated	How It Is Scored
Valuation	25%	P/E ratio, P/B ratio, Forward P/E vs Trailing P/E, PEG ratio	Each metric is scored using percentile ranking against the market. Low P/E = high score (undervalued). PEG < 1 = growth at reasonable price. Forward P/E < Trailing P/E = expected earnings growth.
Profitability	25%	ROE, profit margins, ROA	Metrics ranked into tiers: ROE > 20% = excellent, 15-20% = good, 10-15% = average, <10% = poor. Each tier maps to a score. Higher profitability indicates a company with competitive advantages.
Growth	20%	Revenue growth rate, earnings growth rate	Year-over-year growth rates scored: >20% = excellent, 10-20% = good, 0-10% = moderate, <0% = declining. Growth is weighted less than valuation and profitability because growth alone without profitability destroys value.
Financial Health	20%	Current ratio, debt-to-equity ratio, interest coverage	Current ratio > 2 = strong, 1.5-2 = adequate, <1.5 = concerning. D/E < 0.5 = conservative, 0.5-1 = moderate, >1 = leveraged. Interest coverage > 5 = safe, 3-5 = adequate, <3 = risky.
Dividends	10%	Dividend yield, payout ratio sustainability	Yield > 3% with payout ratio < 60% = sustainable. Very high yield (>8%) with high payout ratio (>80%) may indicate an unsustainable dividend. Weighted lowest because not all growth companies pay dividends.

15. Backend Module: PatternDetector.py - Chart Pattern Detection

This module (47KB) detects 35+ chart patterns across three categories using algorithmic pattern recognition:

- **19 Candlestick Patterns:** Doji (indecision), Hammer (bullish reversal), Inverted Hammer, Engulfing (bullish/bearish), Morning/Evening Star, Marubozu (strong momentum), Harami (inside bar), Three White Soldiers (strong bullish), Three Black Crows (strong bearish), and more. Each pattern is detected by analyzing OHLC relationships of 1-3 consecutive candles.
- **14 Chart Patterns:** Double Top/Bottom, Head and Shoulders, Inverse H&S, Ascending/Descending/Symmetric Triangles, Rising/Falling Wedges, Ascending/Descending Channels, Flags, Pennants, Cup and Handle. Detection uses pivot point identification and trendline fitting via linear regression.
- **4 Trend Patterns:** Golden Cross, Death Cross, Breakout Up/Down. Detected via moving average crossover analysis.
- **Cross-Pattern Confluence Scoring:** When multiple patterns agree (e.g., Double Bottom + Bullish Engulfing + Volume Surge), the confluence score increases, providing higher-confidence signals.
- **Multi-Timeframe Scanning:** Patterns are detected across 5, 10, 20, and 50-day lookback windows to capture signals at different time horizons.
- **Support/Resistance Clustering:** Aggregates price levels where multiple patterns converge, identifying key support and resistance zones.

16. Other Backend Modules

Module	Class	Size	Detailed Description
PortfolioManager.py	PortfolioManager	6.8 KB	Manages user portfolios with transaction-based tracking. Supports BUY/SELL transactions, automatic recalculation of average cost basis, P&L; computation with current market prices, and portfolio summary with total value and return. Persists data as per-user JSON files. Watchlist management allows users to track stocks of interest without holding positions.
UserManager.py	UserManager	8.8 KB	Handles user authentication and account management. Supports local registration (username/email/password with werkzeug password hashing) and Google OAuth (verifies token via googleapis, creates or links accounts). Stores users in PostgreSQL with migration-safe schema evolution (ALTER TABLE ADD COLUMN IF NOT EXISTS). Generates JWT tokens for session management.
ExportManager.py	ReportGenerator	18KB	Generates multi-format reports. CSV exports for screener results, backtest trades, risk metrics, and fundamentals. JSON export for machine consumption. HTML export creates self-contained intelligence dossiers with embedded dark-theme CSS, color-coded risk indicators, AI prediction displays, and regulatory disclaimers. PDF export via ReportLab.
ExchangeManager.py	ExchangeManager	11KB	Unified abstraction layer for multiple stock exchanges. Supports NSE (.NS suffix), BSE (.BO), NYSE, NASDAQ, LSE (.L). Provides exchange-specific configuration: trading hours, timezone, currency, holiday calendars, and market status checking (open/closed). The abstract ExchangeDataProvider interface allows plugging in different data providers.
AdvancedStrategyEngine.py	AdvancedStrategyEngine	42KB	Implements advanced quantitative strategies beyond the basic screener: (1) Pairs Trading using Engle-Granger cointegration test, (2) Statistical Arbitrage with z-score mean reversion, (3) Regime-based strategies using Hidden Markov Models, (4) Factor-based strategies (momentum, value, quality, size, volatility factors). Also includes SectorRotationDetector for detecting capital flows across sectors.
AdvancedFeatureEngine.py	AdvancedFeatureEngine	37KB	Extended feature engineering: wavelet decomposition (multi-resolution analysis), Fourier transform (frequency-domain features), entropy features (Shannon entropy for regime detection), fractal features (fractal dimension), cross-asset features (correlation with NIFTY 50, India VIX, USD/INR, Brent Crude), and market microstructure proxies. Uses information-theoretic feature selection (mutual information scoring).
MultiStrategyScreener.py	MultiStrategyScreen	16KB	Orchestrates running ALL screening strategies across the full stock universe. Uses ThreadPoolExecutor for parallel execution. Supports async job submission with polling: POST to submit, GET to poll status. Maximum 8 strategies per run. Result combination uses weighted scoring.
drift_monitor.py	FeatureDriftMonitor	4.2 KB	Detects distribution shift between training data and live data using Population Stability Index (PSI). $PSI = \sum((actual\% - expected\%) * \ln(actual\%/expected\%))$ for each bin. $PSI < 0.10$ = stable, $0.10-0.25$ = moderate (monitor), ≥ 0.25 = severe (retrain). Runs on each prediction to catch degradation early.

Module	Class	Size	Detailed Description
walkforward.py	PurgedWalkForwardSplitter	12K B	Model-agnostic walk-forward cross-validation. Implements purged splitting with embargo: train_days=504, test_days=63, embargo_days=10. The purge/embargo gap prevents label leakage between train and test sets. Decoupled from any specific model via train_fn and predict_fn callbacks.
redis_client.py	RedisClient (Singleton)	4K B	Singleton wrapper around Redis with: auto-serialization via pickle, configurable TTL, graceful fallback when Redis is unavailable, and @redis_cache decorator that generates MD5-hashed cache keys from function name + arguments.

17. Frontend Components (14 Components)

Each React component is a self-contained UI module with its own state, API calls, and rendering logic. Components are organized into four directories: auth/ (login), charts/ (stock charts), common/ (shared UI), and dashboard/ (main features).

Component	Size	Detailed Functionality
Dashboard.jsx	51KB	The main landing page after login. Displays: (1) Market Overview with key index prices, (2) User's Watchlist with real-time prices and mini sparkline charts, (3) Quick Stock Search with autocomplete, (4) Stock Summary Cards showing price, change%, volume, and key metrics, (5) News Feed with latest market headlines, (6) Quick Action buttons linking to screener, AI analysis, backtest, etc. Uses useState for managing watchlist state and useEffect for fetching market data on mount.
StrategyBuilder.jsx	46KB	The most complex component. Provides a visual interface for building custom screening strategies without coding. Users define: (1) strategy name and description, (2) rules using AND/OR logic operators, (3) each rule specifies an indicator (RSI, MACD, etc.), a comparison operator (>, <, =, crosses above), and a threshold value. Built rules are serialized to JSON and saved via the API. Includes inline backtesting to validate the strategy on historical data before saving. Strategy validation ensures all rules are syntactically correct.
FundamentalDashboard.jsx	37KB	Displays comprehensive fundamental analysis. Shows: (1) Piotroski F-Score with visual breakdown of all 9 criteria (green check for pass, red X for fail), (2) Proprietary composite score as a progress bar (0-100) with category breakdowns, (3) Individual financial metrics with explanatory tooltips, (4) Side-by-side comparison table for evaluating multiple stocks across all fundamental dimensions simultaneously.
ScreenerDashboard.jsx	37KB	Interface for running stock screening strategies. Users: (1) select from 18+ built-in strategies organized by category, (2) configure parameters (confidence threshold, minimum consensus), (3) run the screen and view results in a sortable, filterable table, (4) see per-stock verdict with confidence score, signal type, and which specific conditions were met. Results are color-coded: green for bullish, red for bearish, yellow for neutral.
BacktestRunner.jsx	36KB	Strategy backtesting interface. Users: (1) select a strategy and date range, (2) configure backtest parameters (initial capital, position sizing, commission), (3) run the backtest and view results. Displays: equity curve chart (Recharts), trade log table with entry/exit details, performance metrics (Sharpe, max drawdown, win rate), and comparison against buy-and-hold. Walk-forward mode shows per-fold metrics.
RiskDashboard.jsx	26KB	Displays all 30+ risk metrics organized by category with visual cards. VaR and CVaR are shown as bar charts. Drawdown history as an area chart. Risk-adjusted return metrics as a comparison table. Monte Carlo simulation results as a scatter plot (return vs. volatility) with the efficient frontier curve. Hurst exponent displayed with interpretation (trending/random/mean-reverting).
Navbar.jsx	24KB	Full navigation bar with: route links with active highlighting, Lucide React icons for each section, health status indicator (pings /api/health and shows green/red dot), user avatar and profile display, logout button, and mobile-responsive hamburger menu that transforms to a slide-out drawer on small screens.
SentimentDashboard.jsx	23KB	News sentiment analysis display. Shows: (1) overall sentiment gauge (-1 to +1 with color coding), (2) sentiment trend chart over time, (3) individual news articles with per-article sentiment score, (4) source-level sentiment breakdown (which news sources are most bullish/bearish), (5) keyword cloud of most frequent topics.
PortfolioDashboard.jsx	23KB	Portfolio management interface. Shows: (1) Holdings table with ticker, quantity, average cost, current price, P&L, (2) Allocation pie chart (Recharts), (3) Portfolio-level metrics (total value, total return, daily change), (4) Add/remove position forms, (5) Portfolio risk summary derived from RiskAnalytics.

Component	Size	Detailed Functionality
AIAnalysis.jsx	20KB	AI prediction display. Shows: (1) 5-target prediction results with confidence scores, (2) Direction arrow (bullish/bearish/neutral with color), (3) Volatility and risk badges, (4) Trend strength progress bar, (5) Optimal action recommendation, (6) Model training trigger button, (7) Prediction history table, (8) Model accuracy metrics.
StockChart.jsx	20KB	Full OHLCV candlestick/line chart using Chart.js. Features: (1) Candlestick rendering with green/red coloring, (2) Volume bars as overlay, (3) Technical indicator overlays (moving averages, Bollinger Bands), (4) Time-series x-axis with date formatting via chartjs-adapters-date-fns, (5) Zoom and pan capabilities, (6) Multiple timeframe selection (1W, 1M, 3M, 6M, 1Y, 5Y).
ExportPanel.jsx	17KB	Data export interface. Users select: (1) what to export (screener results, backtest data, risk metrics, fundamentals), (2) export format (HTML report, JSON, CSV), (3) trigger the export and download the file. HTML reports are self-contained intelligence dossiers with professional formatting.
PriceTargets.jsx	11KB	Displays ML-generated price targets for individual stocks. Shows: predicted price target, current price, upside/downside percentage, support and resistance levels, and model confidence. PriceTargetsDashboard.jsx extends this for batch display of multiple stocks.
LoginPage.jsx	8.8KB	Authentication page with Google OAuth integration. Features: animated gradient background using CSS keyframes, feature showcase cards highlighting platform capabilities, Google Sign-In button via useGoogleLogin hook, and Framer Motion entrance animations for a polished first impression.

18. Frontend Services, Routing, and Design System

18.1 API Service Layer (services/api.js - 18KB): All API communication is centralized in a single file. An Axios instance is created with a base URL (from environment variable or defaulting to same-origin proxy). Two interceptors are configured: (1) Request interceptor: automatically reads the JWT token from localStorage and attaches it as an Authorization: Bearer header to every outgoing request. (2) Response interceptor: catches 401 Unauthorized responses, clears the stored token, and redirects to /login. Service functions are organized by domain: authService, stockService, screenerService, predictionService, backtestService, portfolioService, riskService, fundamentalService, sentimentService, exportService, schedulerService, healthService. Each function wraps an axios.get() or axios.post() call with the appropriate endpoint URL and parameters.

18.2 Routing Architecture: React Router v7 provides client-side routing. The App.jsx file defines all routes within a BrowserRouter. A PrivateRoute wrapper component checks if the user is authenticated (token exists in state). If not, it redirects to /login using the Navigate component. If authenticated, it renders the requested component. All protected routes share a common layout with the Navbar component.

18.3 Authentication Flow: (1) User clicks Google Sign-In on LoginPage, (2) @react-oauth/google opens Google's consent screen, (3) Google returns a credential JWT to the frontend, (4) Frontend sends this credential to POST /api/auth/google, (5) Backend verifies the token with Google's servers using google-auth library, (6) Backend creates/links the user account and generates a local JWT session token, (7) Frontend stores the JWT in localStorage and decodes it with jwt-decode to display user info, (8) All subsequent API calls include the JWT via the Axios interceptor.

18.4 Design System (styles/dashboard.css - 23KB): The design system uses CSS custom properties (variables) defined in index.html for consistent theming across the application. The dark theme uses: background #0a0e17 (deep navy), surface #1e293b (slate), primary #6366f1 (indigo), accent #22d3ee (cyan), success #22c55e (green), warning #f59e0b (amber), danger #ef4444 (red). Glassmorphism effects use backdrop-filter: blur(12px) with semi-transparent backgrounds (rgba) to create depth. Typography uses Google Fonts Inter (400, 500, 600, 700 weights) for a clean, modern look. Components use card-based layouts with subtle borders and shadows. Responsive design is achieved via CSS Grid for page layouts and Flexbox for component internals, with media queries for tablet (<768px) and mobile (<480px) breakpoints.

19. Database Schema and Design

PostgreSQL 14+ with TimescaleDB extension. All tables use the public schema with proper indexing and constraints.

Table	Key Columns	Indexes	Detailed Purpose
nse_stocks	date, ticker, open, high, low, close, adj_close, volume	UNIQUE(date, ticker); TimescaleDB hypertable on date	Stores daily OHLCV candlestick data for every stock. The (date, ticker) unique constraint enables the upsert pattern. Converted to a TimescaleDB hypertable for automatic time-based partitioning.
features_engineered	symbol, date, rsi_14, macd, macd_signal, bb_upper, bb_lower, adx, atr, obv, ... (150+ columns)	INDEX on (symbol, date)	Stores all computed technical indicators. Each row corresponds to one stock on one date. This table is rebuilt daily by the feature engineering pipeline. Having features pre-computed avoids recomputing them on every API request.
users	id (SERIAL PK), username, email, password_hash, auth_provider, google_id, avatar_url, created_at	UNIQUE(user name), UNIQUE(email)	User accounts supporting both local and Google OAuth authentication. Passwords are hashed with werkzeug security functions. Schema evolution handled via ADD COLUMN IF NOT EXISTS.
strategies	name, user_id, rules_json, created_at	INDEX on user_id	Custom strategy definitions created by users via the StrategyBuilder. Rules are stored as JSON, enabling flexible rule structures without schema changes.
watchlists	user_id, symbol, added_at	COMPOSITE(user_id, symbol)	User watchlists for tracking stocks of interest. Many-to-many relationship between users and stocks.
portfolio_positions	user_id, symbol, quantity, avg_price	INDEX on user_id	Portfolio holdings tracking. Average price is recalculated on each buy transaction.
screener_results	symbol, strategy, signal, confidence, timestamp	INDEX on (strategy, timestamp)	Caches screening results to avoid re-running expensive screens. TTL-based invalidation ensures freshness.
model_metrics	ticker, accuracy, loss, epoch, timestamp	INDEX on ticker	Tracks ML model performance over time. Used by the drift monitor and model registry for version management.

20. Authentication and Security Architecture

- **Google OAuth 2.0 Flow:** The application delegates authentication to Google, eliminating the need to handle passwords directly. The flow is: user clicks Google Sign-In -> Google displays consent screen -> user grants access -> Google issues a signed ID token (JWT) -> frontend sends this to backend -> backend verifies the token's digital signature against Google's public keys (rotated regularly) -> extracts user claims (email, name, picture) -> creates or links local account -> issues a local session JWT.
 - **JWT (JSON Web Tokens):** Stateless session tokens containing: user ID, email, issue time, expiration (24 hours). Signed with HMAC-SHA256 using a server-side secret key. The stateless nature means any server instance can validate any token without shared session storage, enabling horizontal scaling. Tokens are sent in the HTTP Authorization header as 'Bearer {token}'.
 - **Password Security:** For local accounts, passwords are hashed using `werkzeug.security.generate_password_hash()` which uses PBKDF2 with SHA-256 and a random salt. Verification uses `check_password_hash()`. Plain-text passwords are never stored.
 - **Rate Limiting:** Flask-Limiter prevents brute-force attacks and API abuse. Configured with sliding window: 200 requests/day and 50 requests/hour per IP. Login endpoints have stricter limits.
 - **Security Headers:** Every response includes: X-Content-Type-Options: nosniff (prevents MIME sniffing), X-Frame-Options: DENY (prevents clickjacking), Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy (disables unused browser features).
 - **CORS:** Cross-Origin Resource Sharing is restricted to specific allowed origins (localhost:3000, localhost:5173) rather than wildcard (*), preventing unauthorized domains from making API calls.
-

21. Scheduling and Automation Pipeline

- **APScheduler Configuration:** The SchedulerManager class wraps APScheduler with: cron trigger for daily execution at 4:00 PM IST (16:00 Asia/Kolkata timezone), weekday-only execution (Monday through Friday), NSE holiday calendar awareness (hardcoded holiday dates for 2024-2025 to skip non-trading days), automatic retry with exponential backoff on failure, job history logging.
 - **Daily Pipeline Sequence:** (1) Fetch latest OHLCV data from Yahoo Finance for all ~500 NSE stocks in batches of 50, (2) Upsert new data into PostgreSQL, (3) Run feature engineering to compute 150+ indicators for all stocks, (4) Store engineered features in the `features_engineered` table, (5) Optionally trigger model retraining if auto-retrain is enabled. The entire pipeline typically completes in 15-30 minutes.
 - **Celery Beat Schedule:** In addition to APScheduler, Celery Beat provides: feature engineering at 4:00 PM IST weekdays, sentiment pre-caching at 4:30 PM IST weekdays, full model retraining at 2:00 AM IST every Saturday.
 - **API Control:** The scheduler can be started, stopped, triggered manually, and rescheduled via REST endpoints (`/api/scheduler/status`, `/api/scheduler/trigger`, `/api/scheduler/schedule`).
-

22. Caching and Distributed Task Queue

- **Redis Caching Strategy:** Market data (TTL: 5 minutes during market hours, 1 hour after close), screening results (TTL: 15 minutes), ML predictions (TTL: 1 hour), sentiment scores (TTL: 10 minutes). The `@redis_cache` decorator generates cache keys by hashing `function_name + arguments` with MD5. Cache invalidation happens via TTL expiry or manual delete. The RedisClient singleton provides graceful fallback - if Redis is down, the application continues without caching (slower but functional).
- **Celery Task Queue Architecture:** Redis serves as both the message broker (receives task messages from the API) and the result backend (stores task return values). Celery workers are separate processes that consume tasks from the queue. Task configuration: model training has max 1 retry with 300-second backoff; feature engineering has max 3 retries with 60-second backoff. Task serialization uses JSON. Results expire after 1 hour.

- **Why Async Tasks:** Model training for a single stock takes 1-10 minutes depending on data size and GPU availability. Running this synchronously would block the API and timeout the HTTP connection. By offloading to Celery, the API returns immediately with a task ID, and the frontend can poll for completion.

23. Deployment and Containerization

- **Dockerfile:** Based on python:3.11-slim (minimal Debian image). Installs system dependencies (gcc, libpq-dev for PostgreSQL). Creates directories for saved models, logs, and outputs. Installs Python requirements. Production entrypoint: `gunicorn --bind 0.0.0.0:5000 --workers 4 --threads 2`. Health check: `curl http://localhost:5000/api/health` every 30 seconds.

- **Docker Compose (5 services):** (1) backend: Flask API on Gunicorn with volumes for models/logs, (2) db: timescale/timescaledb:latest-pg15 with persistent volume and health check (`pg_isready`), (3) redis: redis:7-alpine with persistent volume and health check (`redis-cli ping`), (4) celery_worker: same image as backend running '`celery -A celery_app.celery worker`', (5) celery_beat: same image running '`celery -A celery_app.celery beat`'. All services are on a bridge network (`artha_network`) for inter-service DNS resolution. Dependency ordering ensures database and Redis start before the backend.

24. Complete API Reference Summary

The application exposes 90+ REST endpoints. Below is a comprehensive reference:

Group	Endpoints	A u t h	Description
Health	GET /api/health, /health/live, /health/ready, /health/model, /stats	N o	System health and readiness probes
Auth	POST /api/auth/google, /auth/login, /auth/register, /auth/refresh	N o	Authentication and token management
Market Data	GET /api/stocks, /history/{t}, /quote/{t}, /quotes/batch, /market/movers, /market/overview, /stocks/search	J W T	Stock data, quotes, search, market overview
Screening (12+)	POST /api/screen/momentum, /piotroski, /swing, /breakout, /value, /garp, /mean_reversion, /quality_dividend, /trend_following, /contrarian, /quality_growth, /macd_triple_alignment, /custom/run/{name}	J W T	Run individual or custom screening strategies
Multi-Screen	POST /api/screener/multi/run, /multi/jobs; GET /multi/jobs/{id}	J W T	Run multiple strategies with consensus scoring
Predictions	GET /api/predict/{t}, /price-target/{t}, /price-target/{t}/levels; POST /price-targets/batch	J W T	AI neural network predictions and price targets
Training	POST /api/train/{ticker}; GET /api/model/info	J W T	Trigger model training, view model info
Backtesting	POST /api/backtest/{strategy}, /compare, /walk-forward, /custom/{name}; GET /regimes/{t}	J W T	Strategy backtesting and walk-forward validation
Portfolio	GET/POST/DELETE /api/portfolio/watchlist, /holdings, /summary, /add, /remove	J W T	Watchlist and portfolio CRUD
Risk	GET /api/risk/{ticker}; POST /risk/portfolio, /risk/compare, /risk/frontier	J W T	30+ risk metrics, portfolio VaR, efficient frontier
Fundamental	GET /api/fundamental/{t}, /piotroski/{t}, /score/{t}; POST /compare	J W T	Piotroski F-Score, composite score, comparison
Sentiment	GET /api/sentiment/{ticker}, /history, /sources	J W T	News sentiment scores and trends
Export	POST /api/export/report, /csv, /json, /html	J W T	Multi-format report generation and download
Scheduler	GET /api/scheduler/status; POST /trigger, /schedule	J W T	Pipeline scheduler control

Group	Endpoints	A u t h	Description
Data Quality	GET /api/data-quality/report, /reconcile, /stocks/gaps/{t}, /features/quality-report	J W T	Data completeness and quality monitoring
News	GET /api/news/latest, /news/{ticker}	J W T	Market and stock-specific news
Recommen dations	GET /api/recommendations	J W T	AI-generated stock recommendations

25. Design Patterns Applied

The application systematically applies 10+ well-known software design patterns:

Pattern	Where Applied	Detailed Explanation
MVC (Model -View-Controller)	Flask routes (Controller), DB/models (Model), React (View)	Separates concerns: React components handle presentation (View), Flask route handlers process requests and coordinate logic (Controller), database tables and Python data classes represent the domain (Model). Changes to the UI don't affect business logic and vice versa.
App Factory	<code>create_app()</code> in <code>application.py</code>	Flask application is created inside a factory function rather than at module level. This enables: creating multiple app instances for testing, configuring the app differently per environment (dev/test/prod), and lazy initialization of extensions.
Repository Pattern	<code>IntegratedPostgreSQL</code> abstracts DB access	All database operations are encapsulated in the <code>NSEDataPipeline</code> class. Business logic modules (screener, predictor, etc.) never write SQL directly - they call repository methods. This decouples business logic from storage implementation, making it possible to swap PostgreSQL for another database without changing business logic.
Strategy Pattern	Each screening strategy is an independent evaluator	Each screening strategy implements the same interface (evaluate conditions, return verdict) but with different logic. New strategies can be added without modifying existing code (Open/Closed Principle). The <code>StrategyLibrary</code> acts as a registry of available strategies.
Facade Pattern	<code>UnifiedStockPredictor</code> wraps the ML pipeline	The complex ML subsystem (model loading, feature preparation, inference, confidence calibration, safety checks, drift monitoring) is hidden behind a simple <code>predict(ticker)</code> interface. External code doesn't need to understand PyTorch internals.
Observer Pattern	<code>DriftMonitor</code> watches model metrics; <code>SocketIO</code> pushes updates	The drift monitor observes prediction quality metrics and triggers alerts when degradation is detected. <code>SocketIO</code> implements publish-subscribe for real-time quote streaming - the server publishes updates, clients subscribe to receive them.
Builder Pattern	<code>StrategyBuilder</code> constructs strategies from rules	Custom strategies are built step-by-step: add name, add conditions one at a time with AND/OR logic, validate, and finalize. The builder ensures only valid strategies can be created.
Singleton Pattern	<code>Config</code> , <code>RedisClient</code> , DB connection pool, model instances	Ensures exactly one instance of expensive resources. The <code>RedisClient</code> singleton prevents creating multiple connections. Model instances are loaded once and reused. <code>Config</code> is read once at startup.
Decorator Pattern	<code>@jwt_required</code> , <code>@cache.cached</code> , <code>@limiter.limit</code> , <code>@redis_cache</code>	Cross-cutting concerns (authentication, caching, rate limiting) are implemented as decorators that wrap route handler functions. This keeps business logic clean and concerns separated. Decorators can be composed.
Pipeline Pattern	Data -> Features -> Model -> Prediction	Data processing flows through a sequence of stages, each transforming the data for the next stage. The daily pipeline: fetch raw data -> compute features -> (optional) train model -> serve predictions. Each stage is independently testable.
Circuit Breaker	<code>ModelDegradationCircuitBreaker</code>	If the ML model's accuracy drops below a threshold, the circuit breaker 'opens' and stops serving predictions, preventing bad signals from reaching users. After a cooldown period, the circuit breaker allows a few test predictions to check if the model has recovered.
Registry Pattern	<code>ProductionModelRegistry</code> , <code>StrategyLibrary</code>	Maintains a catalog of available models/strategies with metadata. Models are registered with version, training date, and metrics. Strategies are registered with name, category, and rule set. Enables runtime discovery and selection.

26. Machine Learning Concepts Explained

Concept	Detailed Explanation with Mathematical Intuition
LSTM (Long Short-Term Memory)	A recurrent neural network variant invented to solve the vanishing gradient problem. Standard RNNs struggle to learn long-range dependencies because gradients diminish exponentially during backpropagation through time. LSTMs add a cell state (a 'conveyor belt' of information) controlled by three gates: (1) Forget gate: decides what to discard from cell state ($\text{sigmoid} * \text{cell}$), (2) Input gate: decides what new information to store ($\text{sigmoid} * \tanh$), (3) Output gate: decides what to output from cell state ($\text{sigmoid} * \tanh(\text{cell})$). The cell state allows gradients to flow unchanged across many time steps.
Self-Attention Mechanism	Computes $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) * V$ where Q (Query), K (Key), V (Value) are learned linear projections of the input. For each time step, the attention score measures how much to 'attend to' every other time step. The softmax ensures weights sum to 1. The $\sqrt{d_k}$ scaling prevents extremely large dot products that would push softmax into saturation. Multi-head attention runs this process h times in parallel with different projections, capturing different types of relationships.
Temporal Convolutional Network (TCN)	1D convolutions applied along the time axis with dilation. Standard convolution with kernel size k sees k consecutive time steps. Dilated convolution with dilation d sees k time steps spaced d apart, giving an exponentially larger receptive field. With dilation rates $\{1,2,4,8\}$ and kernel size 3, the receptive field grows from 3 to 45 time steps. Causal padding ensures the model cannot see future data. TCNs are parallelizable (unlike sequential RNNs) and capture hierarchical temporal features.
Sinusoidal Positional Encoding	Injects position information into the model: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)})$, $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{(2i/d)})$. Each position gets a unique encoding. The sinusoidal form allows the model to learn relative positions because $\text{PE}(\text{pos}+k)$ can be expressed as a linear function of $\text{PE}(\text{pos})$. This comes from the Transformer architecture.
Multi-Task Learning	Training one model to predict multiple related targets simultaneously. The shared encoder learns representations useful for all tasks. Task-specific heads specialize. Benefits: (1) more data-efficient (each task provides additional training signal), (2) regularization (prevents overfitting to any single task), (3) learned task correlations (knowing volatility helps predict risk level).
Focal Loss	Modification of cross-entropy that down-weights well-classified examples: $\text{FL}(p) = -(1-p)^\gamma * \log(p)$. When $\gamma=2$, examples the model already classifies correctly (p near 1) contribute very little to the loss. This focuses training on hard, misclassified examples and is especially useful when classes are imbalanced (e.g., more Neutral days than Strong Buy days).
Early Stopping	Monitors validation loss during training. If loss doesn't improve for 'patience' epochs (e.g., 10), training stops and the best model is restored. Prevents overfitting: after a point, the model starts memorizing training data noise rather than learning generalizable patterns. The gap between training loss (decreasing) and validation loss (increasing) indicates overfitting.
Batch Normalization	Normalizes layer inputs across the mini-batch: $\hat{x} = (x - \text{mean}) / \sqrt{\text{var} + \text{eps}}$, then scales and shifts: $y = \gamma * \hat{x} + \beta$. Benefits: (1) enables higher learning rates (normalized inputs have well-behaved gradients), (2) reduces internal covariate shift (each layer sees consistent input distributions), (3) acts as a regularizer (batch statistics add noise). γ and β are learned parameters.
Dropout / Spatial Dropout	Standard dropout randomly zeroes individual neurons with probability p during training, forcing the network to not rely on any single neuron. Spatial dropout drops entire feature channels (zeroes an entire feature across all time steps). At test time, dropout is disabled and outputs are scaled by $(1-p)$. This ensemble effect (averaging exponentially many sub-networks) reduces overfitting.

Concept	Detailed Explanation with Mathematical Intuition
Population Stability Index (PSI)	Measures distribution shift between a reference (training) and current dataset. $PSI = \sum((actual_pct - expected_pct) * \ln(actual_pct / expected_pct))$ for each histogram bin. $PSI < 0.10$ = negligible shift, $0.10-0.25$ = moderate (investigate), ≥ 0.25 = significant (retrain). Used by the drift monitor to detect when market conditions have changed enough to invalidate the model.
Kelly Criterion	Optimal bet sizing formula: $f^* = (b*p - q) / b$ where b = net odds (reward/risk ratio), p = probability of winning, $q = 1-p$ = probability of losing. The full Kelly fraction maximizes long-run geometric growth rate of capital. In practice, fractional Kelly (25-50%) is used because: (1) probability estimates are imperfect, (2) full Kelly produces extreme drawdowns, (3) fractional Kelly sacrifices little growth for much less volatility.

27. Financial Concepts and Algorithms Explained

Concept	Detailed Explanation
Value at Risk (VaR)	The maximum expected loss over a specified time period at a given confidence level. For example, '95% daily VaR = 2.5%' means: there is a 95% probability that the stock will not lose more than 2.5% in a single day, or equivalently, there is a 5% chance of losing more than 2.5%. Parametric VaR assumes returns are normally distributed: $VaR = \mu + z_{\alpha} * \sigma$. Historical VaR uses actual return percentiles. Limitation: VaR says nothing about the magnitude of losses beyond the threshold.
Conditional VaR (CVaR / Expected Shortfall)	Answers the question VaR cannot: 'When VaR is breached, how bad are losses expected to be?' $CVaR = E[Loss Loss > VaR]$ = average of all losses exceeding VaR. CVaR is always $\geq VaR$. Example: if 95% VaR = 2.5% and CVaR = 4.1%, then on the 5% worst days, losses average 4.1%. CVaR is considered a superior risk measure because it is coherent (satisfies sub-additivity: portfolio CVaR $\leq$ sum of individual CVaRs).
Sharpe Ratio	Risk-adjusted return: $Sharpe = (R_p - R_f) / \sigma_p$ where R_p = portfolio return, R_f = risk-free rate, σ_p = portfolio standard deviation. Interpretation: return earned per unit of total risk. Sharpe > 1 = good, > 2 = great, > 3 = exceptional. Limitation: treats upside and downside volatility equally - a stock that goes up a lot has high volatility but that's good for investors.
Sortino Ratio	Modification of Sharpe that only penalizes downside risk: $Sortino = (R_p - R_f) / \sigma_{downside}$ where $\sigma_{downside} = \sqrt{\text{mean}(\min(R_i - R_{target}, 0)^2)}$. A stock that goes up a lot (high upside volatility) is not penalized. Generally preferred over Sharpe for asymmetric return distributions.
Piotroski F-Score	A 9-point fundamental scoring system (0-9) developed by Professor Joseph Piotroski. Uses only publicly available financial statement data. Studies show that high F-Score stocks (8-9) significantly outperform low F-Score stocks (0-2). The 9 criteria cover profitability (4 points), leverage/liquidity (3 points), and operating efficiency (2 points).
MACD (Moving Average Convergence Divergence)	Momentum oscillator. MACD Line = 12-period EMA - 26-period EMA. Signal Line = 9-period EMA of MACD Line. Histogram = MACD Line - Signal Line. Bullish signal: MACD crosses above Signal line. Bearish: crosses below. The histogram shows momentum acceleration/deceleration. Created by Gerald Appel in the late 1970s.
RSI (Relative Strength Index)	Momentum oscillator ranging 0-100. $RSI = 100 - (100 / (1 + RS))$ where $RS = \text{average_gain} / \text{average_loss}$ over 14 periods. $RSI > 70$ = overbought (potential reversal down). $RSI < 30$ = oversold (potential reversal up). RSI divergence (price makes new low but RSI doesn't) is a powerful reversal signal. Created by J. Welles Wilder in 1978.
Bollinger Bands	Volatility envelope: Middle Band = 20-period SMA, Upper = Middle + $2 * \sigma$, Lower = Middle - $2 * \sigma$. ~95% of price action falls within the bands. Bollinger Squeeze (narrow bands) indicates low volatility compression that often precedes a big move. Price touching the upper band is not automatically a sell signal - in strong uptrends, price can 'walk the band'.
Ichimoku Cloud	Japanese technical analysis system with 5 lines: Tenkan-sen (9-period midpoint), Kijun-sen (26-period midpoint), Senkou Span A (average of Tenkan and Kijun projected 26 periods forward), Senkou Span B (52-period midpoint projected 26 periods forward), Chikou Span (close projected 26 periods backward). The 'cloud' between Senkou Span A and B provides support/resistance zones.
Monte Carlo Efficient Frontier	Estimates the set of optimal portfolios by randomly generating thousands of portfolio weight combinations. Each random portfolio's expected return and volatility are computed. Plotting return vs. risk for all portfolios reveals the 'efficient frontier' - the curve of portfolios offering the maximum return for each level of risk. Key portfolios: Maximum Sharpe (tangent portfolio) and Minimum Volatility.

Concept	Detailed Explanation
Hurst Exponent	Measures long-term memory in a time series via Rescaled Range (R/S) analysis. $H < 0.5$ = mean-reverting (anti-persistent, deviations tend to be followed by reversals). $H = 0.5$ = random walk (no memory). $H > 0.5$ = trending (persistent, deviations tend to be followed by similar deviations). For trading: mean-reverting stocks favor range strategies; trending stocks favor momentum strategies.

28. Software Engineering Concepts Explained

Concept	Detailed Explanation in Context of Artha Drishti
REST API	REpresentational State Transfer. An architectural style for web services where: each URL represents a resource (e.g., /api/stocks/TCS.NS = the TCS stock resource), HTTP methods define operations (GET = read, POST = create/compute, PUT = update, DELETE = remove), responses are stateless JSON, and the API is uniform and predictable. Flask implements REST by mapping route decorators to Python functions.
JWT Authentication	JSON Web Tokens consist of three base64-encoded parts: Header (algorithm, type), Payload (claims: user_id, email, exp), Signature (HMAC-SHA256 of header+payload with secret key). The server generates the token on login and the client sends it with every request. The server verifies the signature to ensure the token hasn't been tampered with. No server-side session storage is needed, making it ideal for distributed systems.
OAuth 2.0	An authorization framework that allows third-party applications to access user resources without sharing credentials. In Artha Drishti, Google acts as the identity provider. The user authenticates with Google (not with Artha Drishti), and Google issues a token that Artha Drishti can verify. This is more secure because: (1) Artha Drishti never sees the user's Google password, (2) Google handles 2FA, account recovery, etc.
Single Page Application (SPA)	The React app loads a single HTML page and JavaScript bundle. All subsequent navigation is handled by JavaScript (React Router) which swaps components in/out without full page reloads. Benefits: (1) faster navigation (no server roundtrip for page loads), (2) smoother transitions, (3) the app feels like a native application. Drawback: initial load is larger (mitigated by code splitting).
Component Architecture	React decomposes the UI into reusable, isolated components. Each component manages its own state and rendering. Components communicate via props (parent to child) and callbacks (child to parent). This enables: parallel development (different developers can work on different components), reuse (the same chart component is used in multiple dashboards), and testing (each component can be tested in isolation).
Axios Interceptors	Middleware functions that execute on every HTTP request/response. Request interceptors modify outgoing requests (e.g., adding the JWT token header). Response interceptors process incoming responses (e.g., catching 401 errors and redirecting to login). This centralizes cross-cutting concerns instead of duplicating logic in every API call.
Rate Limiting	Controls the rate of API requests to prevent abuse, ensure fair usage, and protect server resources. The sliding window algorithm tracks requests within a moving time window. When the limit is exceeded, the server returns HTTP 429 (Too Many Requests) with a Retry-After header. In production, rate limit counters are stored in Redis for distributed enforcement across multiple server instances.
Distributed Task Queue (Celery)	Decouples request handling from long-running work. Architecture: Producer (Flask API) creates task messages and puts them on a queue (Redis). Consumer (Celery worker process) picks up tasks and executes them asynchronously. Benefits: (1) API responds immediately (no timeout), (2) workers can be scaled independently, (3) failed tasks can be retried automatically, (4) task progress can be tracked via the result backend.
Containerization (Docker)	Packages the application, its dependencies, and its runtime environment into a standardized unit (container). Unlike VMs, containers share the host OS kernel, making them lightweight (MB vs GB). Docker ensures 'works on my machine' is eliminated - if it runs in a container locally, it runs the same way in production. docker-compose orchestrates multi-container applications.
Time-Series Database (TimescaleDB)	Extends PostgreSQL for time-series workloads. Regular PostgreSQL tables degrade in performance as they grow to millions of rows because index updates become expensive. TimescaleDB hypertables automatically partition data into time-based chunks. This enables: (1) fast time-range queries (only relevant chunks are scanned), (2) efficient data lifecycle management (drop old chunks instantly), (3) native compression.

Concept	Detailed Explanation in Context of Artha Drishti
Connection Pooling	Maintaining a pool of pre-established database connections that are reused across requests. Creating a new TCP connection to PostgreSQL takes ~50-100ms (DNS, TCP handshake, authentication). With pooling, connections are borrowed from the pool (sub-millisecond) and returned when done. Configured with: pool_size=10 (normal connections), max_overflow=20 (burst capacity), pool_recycle=3600 (refresh every hour to handle idle timeouts).
Upsert Pattern	A combination of INSERT and UPDATE: INSERT INTO table VALUES (...) ON CONFLICT (unique_key) DO UPDATE SET col = EXCLUDED.col. If the row doesn't exist, it's inserted. If it already exists (conflict on unique key), it's updated. This makes data loading idempotent - running the pipeline twice with the same data produces the same result. Essential for reliability in automated data pipelines.

-- End of Document --